รายงานโครงงาน

รายวิชา Object Oriented Data Structure

ภาคการศึกษา 1/2569

โรงแรมอนันต์แบบกระจาย (Distributed Hilbert's Hotel) ด้วยเทคนิค Consistent Hashing

ชื่อกลุ่ม เยอมรับก็ได้ ว่าเยังรักUอยู่

อาจารย์ผู้สอน ผศ. เกียรติณรงค์ ทองประเสริฐ

จัดทำโดย

ณัฐภูมิ บาดขุนทด รหัสนักศึกษา 68010366

ฑริตา เปรมชุติวัต รหัสนักศึกษา 68010283

ปลิตา ชูรัตน์ รหัสนักศึกษา 68010669

พงศธร ฟักผล รหัสนักศึกษา 68010726

ปริญา บุญสิงมา รหัสนักศึกษา 68010662

สารบัญ

1. โจทย์และขอบเขต 4

1.1 นิยามสัญลักษณ์ 4

2. การออกแบบเชิงวัตถุ 5

2.1 โครงสร้างโมดูลและคลาส 5

2.2 แผนภาพคลาส (Class Diagram) 6

2.3 หน้าที่ของแต่ละคลาส 6

2.4 โครงสร้างข้อมูลที่เลือกและเหตุผล 7

2.5 ตัวอย่างการรักษาข้อมูลหลายชุดให้สอดคล้องกัน 7

3. กติกาการคำนวณ 8

3.1 รหัสแขกและข้อมูลที่จัดเก็บ 8

3.2 หมายเลขห้อง (Cantor pairing) 9

3.3 วงแหวนและการเลือกอาคาร 9

3.4 การเพิ่ม/ลบอาคารและการนับการย้าย 10

3.5 กรณีขอบ (หัวข้อ 4.5) และคำตอบของโปรแกรม 10

4. ฟังก์ชันของโปรแกรม 11

5. การวิเคราะห์ Big-O 12

5.1 ข้อสมมติฐาน 12

5.2 ตารางวิเคราะห์ตามขั้นตอนของโจทย์หัวข้อ 8 13

5.3 พื้นที่หน่วยความจำ (ขั้นตอนที่ 6) 14

5.4 ตัวเลือกอาคารแบบ hash mod N (ใช้เทียบ) 15

6. วิธีทดลอง 15

6.1 พารามิเตอร์ของชุดการทดลอง (หัวข้อ 7.1) 15

6.2 การทำซ้ำและ salt 15

6.3 วิธีวัด 16

6.4 เครื่องและเวอร์ชันที่ใช้ 16

7. ผลการทดลอง 17

7.1 การทดลอง A: ขนาดข้อมูล K (N = 10, V = 32) 17

7.2 การทดลอง B: จำนวนอาคาร N (K = 10,000, V = 32) 20

7.3 การทดลอง C: จำนวนจุดเสมือน V (N = 10, K = 10,000) 23

8. ตารางเปรียบเทียบ hash mod N กับ Consistent Hashing 26
9. การทดสอบความถูกต้อง 27
9.1 ตัวอย่างวงแหวนเล็ก (หัวข้อ 9.1) 27
9.2 ความครอบคลุมของการทดสอบตามโจทย์หัวข้อ 9.2 และ 4.5 28
9.3 ผลการรันชุดทดสอบ 29
10. ข้อจำกัด 31
11. ตารางหน้าที่สมาชิก 32
12. แหล่งอ้างอิง 33

1. โจทย์และขอบเขต

โครงงานนี้ให้เขียนโปรแกรมภาษา Python จัดแขกเข้าพักในเครือโรงแรมที่มีหลายอาคาร โดย “คำนวณ” ว่าแขกอยู่อาคารใดและห้องใด ไม่ใช่จดบันทึกทีละคน อาคารเลือกด้วย Consistent Hashing (วงแหวนแฮชพร้อม virtual node) ส่วนหมายเลขห้องคำนวณจากรหัสแขกโดยตรงด้วย Cantor pairing function เมื่อมีแขกใหม่เข้ามา แขกเดิมต้องไม่ย้ายห้อง และเมื่อเพิ่มหรือลดอาคารต้องมีแขกย้ายน้อยที่สุด โปรแกรมต้องรายงานว่าใครย้ายไปไหน วิเคราะห์ Big-O จากโค้ดจริง และทดลองเทียบกับวิธี hash mod N

ขอบเขต: อาคารแต่ละหลังเป็นวัตถุ (object) ภายในโปรแกรมเดียวที่รันบนเครื่องเดียว ไม่มีระบบเครือข่ายหรือฐานข้อมูล ไม่จำลองเครื่องล่ม ส่วนติดต่อผู้ใช้เป็นเมนูข้อความ และ “ห้องไม่จำกัด” หมายถึงเลขห้องไม่มีเพดานที่กำหนดไว้ล่วงหน้า (Python จำนวนเต็มขยายขนาดได้) ไม่ได้สร้างห้องว่างจริง

1.1 นิยามสัญลักษณ์

สัญลักษณ์	ความหมายในรายงานนี้
N	จำนวนอาคารที่มีอยู่ก่อนการเพิ่มหรือลบอาคารในรอบนั้น ($N \geq 1$)
V	จำนวน virtual node ต่ออาคาร ใช้เท่ากันทุกอาคารและคงที่ภายในการทดลองหนึ่งรอบ
M	จำนวนจุดบนวงแหวนทั้งหมด $M = N \times V$
K	จำนวนแขกที่อยู่ในระบบ ณ เวลาที่วิเคราะห์ (หรือก่อนเปลี่ยนจำนวนอาคาร)
ΔK	จำนวนแขกที่ต้องย้ายอาคารในการเพิ่ม/ลบอาคาร 1 ครั้ง (นับเป็นรหัสแขก ไม่ใช่จำนวน virtual node)
B	จำนวนแขกที่เพิ่มพร้อมกันเป็นกลุ่ม
(c, s)	รหัสแขก: c = ช่องทางที่เดินทางมา, s = ลำดับภายในช่องทางนั้น (จำนวนเต็มบวก)
(node_id, room_no)	ที่อยู่ห้อง: รหัสอาคาร และหมายเลขห้องภายในอาคาร

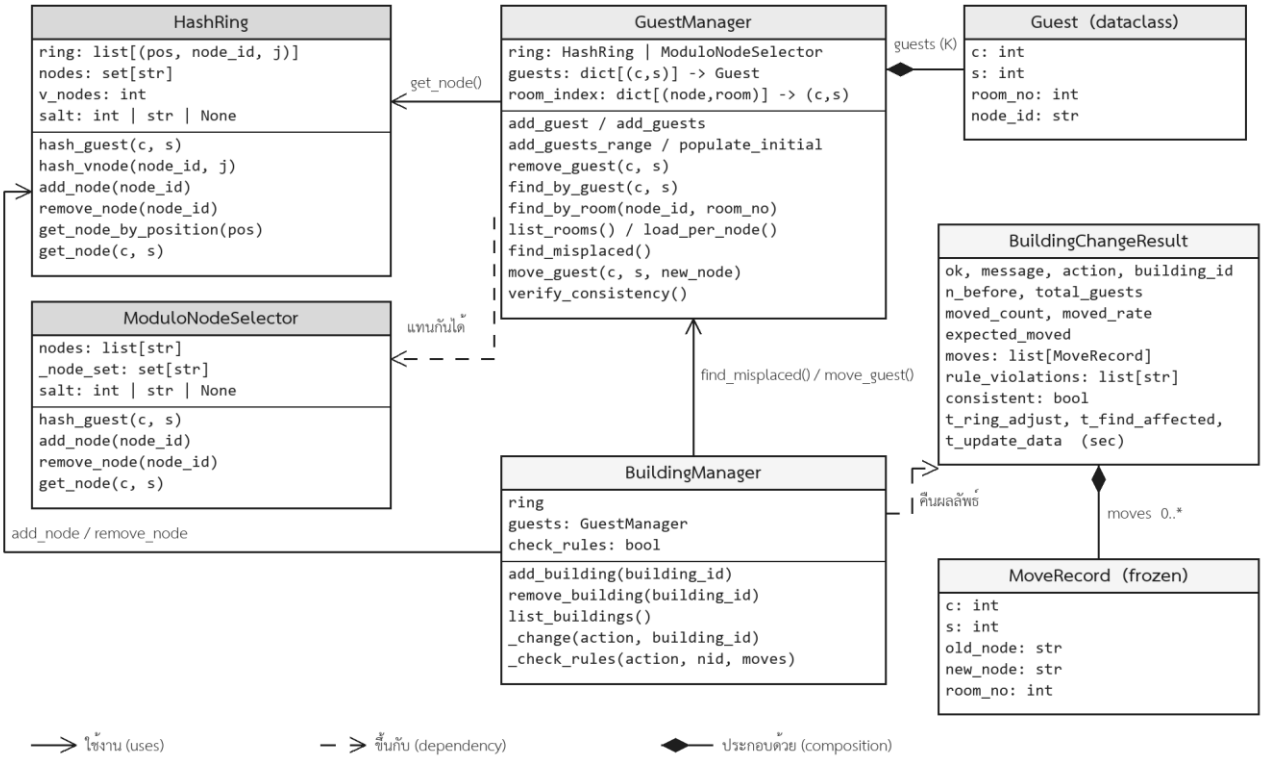
2. การออกแบบเชิงวัตถุ

2.1 โครงสร้างโมดูลและคลาส

โค้ดแบ่งเป็นโมดูลตามหน้าที่ และแยกตรรกะหลัก (คำนวณ จัดข้อมูล ย้ายแขก) ออกจากส่วนเมนูและการแสดงผลโดยสิ้นเชิง คลาสในตรรกะหลักไม่เรียก **`print`** หรือ **`input`** เลย เมธอดส่วนใหญ่คืนค่า (สำเร็จหรือไม่, ข้อความ) หรือวัตถุผลลัพธ์ แล้วให้เมนูเป็นผู้พิมพ์

ไฟล์	คลาส/ฟังก์ชันหลัก	หน้าที่
hash_ring.py	HashRing, ModuloNodeSelector, compute_sha256_64bit	วงแหวน Consistent Hashing (SHA-256 64 บิต + list เรียงลำดับ + bisect) และตัวเลือกอาคารแบบ hash mod N สำหรับเปรียบเทียบ
guest_room.py	cantor_room_no, Guest, GuestManager	สูตรเลขห้อง ข้อมูลแขก ดัชนี 2 ทาง การเพิ่ม/ลบแขก การค้นหา และตรวจความสอดคล้อง
building_manager.py	BuildingManager, MoveRecord, BuildingChangeResult	เพิ่ม/ลบอาคาร หาและย้ายแขก ตรวจกติกาการย้าย นับจำนวนย้าย จับเวลาแยกขั้นตอน
common.py	load_statistics, summarize_rounds, measure_memory	พารามิเตอร์การทดลอง สถิติโหลด (population SD และ CV) และตัววัดหน่วยความจำด้วย tracemalloc
experiments.py / plots.py	run_experiment_A/B/C, plot_experiment_A/B/C	ทดลอง A, B, C ส่งออก CSV และวาดกราฟ (plots.py ต้องใช้ matplotlib)
menu.py	do_init, do_add_building, ... (13 เมนู)	ส่วนติดต่อผู้ใช้แบบเมนูข้อความ เรียกใช้คลาสข้างต้นเท่านั้น
test_all.py	unittest 34 ข้อ	ชุดทดสอบอัตโนมัติ รันซ้ำได้

2.2 แผนภาพคลาส (Class Diagram)



ภาพที่ 1 แผนภาพคลาสของระบบ (จากโค้ดจริง) GuestManager ใช้ตัวเลือกอาคารได้ทั้ง HashRing และ ModuloNodeSelector เพราะทั้งสองมีเมธอด get_node(c, s) เหมือนกัน

2.3 หน้าที่ของแต่ละคลาส

คลาส	ความรับผิดชอบ	สิ่งที่ไม่ทำ
HashRing	เก็บ virtual node M จุดเรียงลำดับ แฮชรหัสแขกและจุดเสมือน ค้นอาคารเจ้าของตำแหน่ง เพิ่ม/ลบอาคารบนวงแหวน	ไม่รู้จักแขกและเลขห้อง
ModuloNodeSelector	ตัวเลือกอาคารแบบ H(guest) mod N ใช้ฟังก์ชันแฮชเดียวกับ HashRing เพื่อเทียบอย่างยุติธรรม	ไม่มี virtual node
Guest (dataclass)	ข้อมูลแขก 1 คน: c, s, room_no (คงที่ตลอด), node_id (เปลี่ยนได้เมื่อย้ายอาคาร)	ไม่มีตรรกะ
GuestManager	เก็บแขกและดัชนี 2 ทาง เพิ่ม/ลบแขก (แบบกลุ่มเป็น all-or-nothing) ค้นหา 2 ทาง แสดงรายการห้อง นับโหลด หาแขกที่อาคารไม่ตรงกับวงแหวน (find_misplaced) ย้ายแขก (move_guest) และตรวจความสอดคล้อง (verify_consistency)	ไม่ปรับวงแหวน
BuildingManager	ประสานการเพิ่ม/ลบอาคาร: ปรับวงแหวน ⟶ หาแขกที่ต้องย้าย ⟶ ตรวจกติกา 4.4 ข้อ 2 ⟶ ย้ายแขก ⟶ ตรวจความสอดคล้อง ⟶ คำนวณอัตราย้ายและจับเวลาแยก 3 ขั้นตอน	ไม่แฮชเอง ไม่พิมพ์ผล
MoveRecord (frozen)	บันทึกการย้าย 1 คน: c, s, old_node, new_node, room_no	แก้ไขไม่ได้หลังสร้าง

คลาส	ความรับผิดชอบ	สิ่งที่ไม่ทำ
BuildingChangeResult	ผลของการเพิ่ม/ลบอาคาร 1 ครั้ง: ok, message, จำนวนย้าย, อัตราย้าย, ค่าคาดหวัง, รายการย้าย, ข้อผิดกติกา, สถานะความสอดคล้อง, เวลา 3 ขั้นตอน	เป็นข้อมูลล้วน

2.4 โครงสร้างข้อมูลที่เลือกและเหตุผล

เลือกจากรูปแบบการใช้งานจริงของโปรแกรม: การ “ค้นอาคารของแขก” เกิดบ่อยที่สุด (ทุกการเพิ่มแขก ทุกการค้นหา และ K ครั้งในทุกการเพิ่ม/ลบอาคาร) ขณะที่การแก้ไขวงแหวนเกิดน้อยและเล็ก (V จุดต่อการเพิ่มอาคาร 1 ครั้ง)

ข้อมูล	โครงสร้าง	รูปแบบการใช้งานจริง	เหตุผล / ราคาที่จ่าย
จุดบนวงแหวน	list[(position, node_id, j)] เรียงลำดับ + bisect	ค้นหาบ่อยมาก แทรก V จุด/การเพิ่มอาคาร ลบทั้งอาคารเป็นครั้งคราว	ค้น O(log M) แบบเรียงลำดับเรียบง่าย การเรียงทูเพิลได้กติกาจัดการแฮชชนกันตามหัวข้อ 4.3 ข้อ 4 โดยอัตโนมัติ ราคาที่จ่าย: แทรกต้องเลื่อนสมาชิก O(M) ต่อจุด ยอมรับได้เพราะ M มีขนาดหลักร้อย และวัดได้เพียงราว 50 µs ต่อการเพิ่มอาคาร (ตารางที่ 6)
ชุดรหัสอาคาร	set[str] (nodes)	ตรวจรหัสซ้ำ/ไม่มีอยู่ทุกครั้งที่เพิ่ม-ลบอาคาร	ตรวจสมาชิก O(1) เฉลี่ย
แขกทั้งหมด	dict[(c, s)] ⟶ Guest	ตรวจรหัสซ้ำ ค้นด้วยรหัส ลบ ทุกครั้งใช้รหัสเป็นกุญแจ	ตรวจ/ค้น/ลบ O(1) เฉลี่ย ถ้าใช้ list ต้องไล่ O(K)
ดัชนีห้อง	dict[(node_id, room_no)] ⟶ (c, s)	ค้นย้อนจากที่อยู่ห้องเป็นรหัสแขก	ค้นย้อน O(1) โดยไม่ต้องไล่แขก K คน ราคาที่จ่าย: เก็บเพิ่ม O(K) และต้องปรับพร้อม guests ทุกครั้งที่เพิ่ม/ลบ/ย้ายแขก
รายการย้าย	list[MoveRecord] (frozen dataclass)	สร้างหลังหาแขกที่ต้องย้าย แล้วเรียงตาม (c, s) เพื่อให้รายงานซ้ำได้	ข้อมูลอ่านอย่างเดียว ป้องกันการแก้ไขโดยไม่ตั้งใจ
hash mod N	list[str] + set[str]	ลำดับในรายการกำหนดดัชนีของ mod จึงต้องรักษาลำดับ; set ไว้ตรวจซ้ำ	พื้นที่ O(N) ไม่ใช่ O(1)

2.5 ตัวอย่างการรักษาข้อมูลหลายชุดให้สอดคล้องกัน

แขกหนึ่งคนถูกเก็บในสองโครงสร้างพร้อมกัน คือ `guests` (รหัส ⟶ ข้อมูล) และ `room_index` (ห้อง ⟶ รหัส) ถ้าอัปเดตเพียงชุดเดียว การค้นด้วยห้องจะผิด เมธอด `move_guest` จึงทำสามขั้นตอนต่อกันเสมอ: ลบกุญแจเดิมใน `room_index` ⟶ เปลี่ยน `node_id` ของ Guest ⟶ ใส่กุญแจใหม่ใน `room_index` โดย `room_no` ไม่เปลี่ยน

ตัวอย่างจริงจาก results/experiment_B_moves.csv (แถวแรกของไฟล์: salt 0, N = 5, V = 32, K = 10,000, เพิ่มอาคาร NODE_NEW):

```
B,0,5,32,10000,consistent,add,NODE_NEW,1,25,NODE03,NODE_NEW,325
```

แขกรหัส (c, s) = (1, 25) มี a = 0, b = 24 จึงได้ room_no = (24 × 25) // 2 + 24 + 1 = 325 อยู่ที่ NODE03 เมื่อเพิ่ม NODE_NEW วงแหวนเปลี่ยน แขกคนนี้เป็น 1 ใน 2,031 คนที่ต้องย้าย สถานะของโครงสร้างทั้งสองก่อนและหลัง (ตรวจโดยรันโค้ดจริงด้วยพารามิเตอร์เดียวกัน ดู report/extra_measurements.py):

โครงสร้าง	ก่อนเพิ่มอาคาร	หลัง move_guest (1, 25, "NODE_NEW")
guests[(1, 25)]	Guest(c=1, s=25, room_no=325, node_id="NODE03")	Guest(c=1, s=25, room_no=325, node_id="NODE_NEW") (room_no เดิม)
room_index	("NODE03", 325) ⟶ (1, 25)	ลบกุญแจ ("NODE03", 325) ออก; เพิ่ม ("NODE_NEW", 325) ⟶ (1, 25)
find_by_room("NODE03", 325)	(1, 25)	None (ห้องนี้ใน NODE03 ว่างแล้ว)
find_by_room("NODE_NEW", 325)	None	(1, 25)
find_by_guest(1, 25)	("NODE03", 325)	("NODE_NEW", 325)

หลังย้ายแขกครบ 2,031 คน `verify_consistency()` ได้ True (จำนวนรายการใน guests = room_index = 10,000; ทุกแขกชี้กลับถึงกัน และ node_id ตรงกับวงแหวนปัจจุบัน) ผลนี้ตรงกับคอลัมน์ consistent = True ทุกแถวใน results/experiment_B_rounds.csv และชุดทดสอบ `test_move_records_and_room_lookup` ตรวจรูปแบบเดียวกันกับแขกทุกคนที่ย้าย

3. กติกาการคำนวณ

3.1 รหัสแขกและข้อมูลที่จัดเก็บ

- แขกระบุด้วย (c, s) เป็นจำนวนเต็มบวก (ไม่รับ 0, ค่าติดลบ, ทศนิยม, สตริง หรือ bool) รหัสต้องไม่ซ้ำ
- ข้อมูลที่เก็บต่อแขก: c, s, room_no (คำนวณครั้งเดียวตอนรับเข้า ไม่เปลี่ยนอีก) และ node_id (อาคารปัจจุบัน เปลี่ยนเมื่อถูกย้าย)
- การเลือกอาคารคำนวณจากวงแหวนปัจจุบันเสมอ ไม่ใช้ตารางคำตอบที่กำหนดไว้ล่วงหน้า (`find_by_guest` เรียก `ring.get_node` สดทุกครั้ง)
- เพิ่มแขกเป็นกลุ่มด้วย (c, s_start, count): count ต้องเป็นจำนวนเต็มไม่ติดลบ (0 = ไม่เปลี่ยนข้อมูล) ถ้ารหัสทับกับแขกเดิมหรือซ้ำกันเองในกลุ่ม ปฏิเสธทั้งกลุ่ม

- นำแขกออก = ลบข้อมูลทั้งสองดัชนี ไม่สร้างห้องว่างทดแทน ถ้าเพิ่มรหัสเดิมกลับเข้ามาถือเป็นการเข้าพักใหม่ที่คำนวณจากวงแหวนปัจจุบัน

3.2 หมายเลขห้อง (Cantor pairing)

ให้ a = c − 1 และ b = s − 1 แล้ว

```
room_no = ((a + b) * (a + b + 1)) // 2 + b + 1      # จำนวนเต็มล้วน ไม่ใช้ทศนิยม
```

ทำไมเลขห้องไม่ซ้ำ: ให้ w = a + b คู่ (a, b) ทั้งหมดที่ a + b < w มีจำนวน 1 + 2 + … + w = w(w + 1)/2 คู่ คู่ที่ a + b = w มี w + 1 คู่ โดย b = 0, 1, …, w แต่ละคู่จึงได้ค่า w(w + 1)/2 + b ที่ต่างกัน อยู่ในช่วง w(w + 1)/2 ถึง w(w + 1)/2 + w ส่วนเส้นทแยงมุมถัดไปเริ่มที่ (w + 1)(w + 2)/2 = w(w + 1)/2 + w + 1 ซึ่งมากกว่าค่าสูงสุดของเส้นนี้พอดี ช่วงของเส้นทแยงมุมต่างๆ จึงไม่ซ้อนกัน เมื่อบวก 1 ได้จำนวนเต็มบวกที่ไม่ซ้ำกันสำหรับทุก (c, s) ที่ต่างกัน (เป็น bijection จากคู่จำนวนเต็มไม่ติดลบไปยังจำนวนเต็มไม่ติดลบ) ดังนั้นเลขห้องขึ้นกับ (c, s) เท่านั้น ไม่ขึ้นกับอาคารและจำนวนแขก แขกใหม่จึงไม่ชนแขกเดิม และเมื่อย้ายอาคารเลขห้องยังเท่าเดิม

รหัสแขก (c, s)	(1, 1)	(2, 1)	(1, 2)	(3, 1)	(2, 2)
a = c − 1, b = s − 1	(0, 0)	(1, 0)	(0, 1)	(2, 0)	(1, 1)
a + b	0	1	1	2	2
room_no	1	2	3	4	5

ตารางนี้ตรงกับตัวอย่างในหัวข้อ 4.2 ของโจทย์ และถูกตรวจโดย `test_known_values`; `test_unique` ตรวจว่า 59 × 59 = 3,481 คู่ได้เลขห้องที่ไม่ซ้ำ 3,481 ค่า (เป็นการตรวจตัวอย่าง ส่วนการพิสูจน์ทั่วไปคือข้อความด้านบน)

3.3 วงแหวนและการเลือกอาคาร

- ตำแหน่งบนวงแหวนเป็นจำนวนเต็ม 0 ถึง 2^64 − 1: แฮชข้อความ UTF-8 ด้วย SHA-256 แล้วนำ 8 ไบต์แรกมาอ่านเป็นจำนวนเต็มแบบ big-endian ไม่มีเครื่องหมาย (`int.from_bytes(digest[:8], "big")`) ไม่ใช้ `hash()` ของ Python เพราะค่าไม่คงที่ข้ามการรัน
- ข้อความที่แฮช: แขก = `guest:{c}:{s}` และจุดเสมือน = `node:{node_id}:{j}` โดย j = 0 ถึง V − 1; เมื่อกำหนด salt จะนำหน้าด้วย `salt:{salt}:` (เช่น `salt:0:guest:1:1` ให้ค่า 2075508245625393561 ตามที่ `test_sha256_fixed_values` ตรึงไว้)
- แขกไปอาคารเจ้าของจุดแรกที่มีตำแหน่ง ≥ ตำแหน่งของแขก ถ้าเลยจุดสุดท้ายให้วนกลับจุดแรกของวงแหวน (โค้ด: `bisect_left(ring, (guest_pos, "", -1))` และถ้า idx เท่ากับความยาว list ให้ idx = 0)

- **การจัดการแฮชชน:** เก็บทุกจุดไว้ใน list และเรียงด้วยคีย์ทูเพิล (position, node_id, j) ดังนั้นจุดที่ position เท่ากันจะเรียงตาม node_id (ตามลำดับตัวอักษร) แล้วตาม j คีย์ค้น (guest_pos, "", -1) ต่ำกว่าทูเพิลจริงทุกตัวที่ position เท่ากัน (node_id ถูกตรวจว่าไม่ว่าง) bisect_left จึงเลือกจุดแรกตามลำดับนั้นเสมอ เช่น ถ้า B มีจุดที่ 50 สองจุดและ A มีจุดที่ 50 หนึ่งจุด แขกที่ตำแหน่ง 50 ไปที่ A (`test_section_9_2_4_search_and_collision`) แขกหลายคนแฮชตรงกันได้ และยังมีเลขห้องต่างกัน เพราะเลขห้องไม่มาจากแฮช

3.4 การเพิ่ม/ลบอาคารและการนับการย้าย

1. เพิ่มอาคาร = แทรกเฉพาะ virtual node ของอาคารใหม่ V จุด ลบอาคาร = เอา virtual node ของอาคารนั้นออกทั้งหมด จุดที่เหลือไม่เปลี่ยนรหัสหรือตำแหน่ง และ V คงที่ตลอดรอบ
2. ตรวจกติกาหลังหาแขกที่ต้องย้าย (`_check_rules`): เพิ่มอาคาร ⟶ ผู้ย้ายต้องเข้าอาคารใหม่เท่านั้น; ลบอาคาร ⟶ ผู้ย้ายต้องมาจากอาคารที่ลบเท่านั้น ถ้าไม่ตรงจะบันทึกใน `rule_violations`
3. จำนวนที่ย้ายนับเป็นรหัสแขกที่ node_id ก่อนและหลังต่างกัน คนละหนึ่งครั้ง (ไม่ใช่จำนวน virtual node) รายการย้ายระบุ (c, s), อาคารเดิม, อาคารใหม่, room_no
4. อัตราย้าย = จำนวนย้าย / K เมื่อ K > 0; ถ้า K = 0 จำนวนย้ายเป็น 0, `moved_rate = None` พร้อมข้อความอธิบายว่าไม่คำนวณเพราะไม่มีแขก
5. ค่าคาดหวัง K/(N + 1) (เพิ่ม) และ K/N (ลบ) เก็บไว้ใน `expected_moved` เพื่อเทียบ เป็นค่าคาดหวังทางสถิติ ไม่ใช่ขอบเขตกรณีเลวร้าย

วิธีหาแขกที่ต้องย้าย: โปรแกรมใช้ **แบบ ก ของโจทย์หัวข้อ 4.7 (ตรวจใหม่ทุกคน)** คือ `GuestManager.find_misplaced()` วนแขกทั้ง K คน คำนวณอาคารใหม่จากวงแหวนแล้วเทียบกับ node_id เดิม จึงมีต้นทุน O(K log M) แม้จะมีผู้ย้ายจริงเพียงไม่กี่คน รายงานนี้ไม่ใช้ตัวเลขของแบบ ข (หาเฉพาะช่วงที่ได้รับผลกระทบ) เพราะโปรแกรมไม่ได้เขียนแบบนั้น

3.5 กรณีขอบ (หัวข้อ 4.5) และคำตอบของโปรแกรม

กรณี	สิ่งที่โปรแกรมทำ (ข้อมูลเดิมคงไว้ ไม่หยุดทำงาน)	ทดสอบโดย
เพิ่มรหัสอาคารที่มีอยู่แล้ว	`HashRing.add_node` คืน (False, "ปฏิเสธคำสั่ง: อาคาร ... มีอยู่ในระบบแล้ว") ก่อนแก้ set หรือ list; `BuildingManager` คืน ok=False และไม่แตะข้อมูลแขก	test_section_4_5_node_edge_cases_and_modulo, test_edge_cases_leave_everything_unchanged
ลบรหัสอาคารที่ไม่มีอยู่	`remove_node` คืน (False, "ไม่พบอาคาร ...") ไม่มี KeyError และไม่มีรายงานการย้าย (moved_count = 0)	test_section_4_5_node_edge_cases_and_modulo, test_edge_cases_leave_everything_unchanged

กรณี	สิ่งที่โปรแกรมทำ (ข้อมูลเดิมคงไว้ ไม่หยุดทำงาน)	ทดสอบโดย
ลบอาคารสุดท้าย (แม้ไม่มีแขก)	ปฏิเสธเพื่อรักษา N ≥ 1 วงแหวนไม่ว่างและยังค้นหาได้	test_9_2_6_remove_last_building_without_guests, test_edge_cases_leave_everything_unchanged
เพิ่มรหัสแขกซ้ำ (เดี่ยว/กลุ่ม)	รายบุคคลปฏิเสธทันที; กลุ่มตรวจทั้งกลุ่มก่อน (รวมซ้ำกันเองในกลุ่ม) ถ้าซ้ำแม้ 1 รายการปฏิเสธทั้งกลุ่ม ไม่เพิ่มใครเลย	test_add_remove_and_duplicates, test_group_all_or_nothing, test_9_2_3_rejected_batch_leaves_data_untouched, test_add_guests_range
นำแขกที่ไม่มีอยู่ออก	`remove_guest` คืน (False, "not found") ไม่ลบ ไม่หักจำนวนแขก สถิติไม่เพี้ยน	test_add_remove_and_duplicates
ค้นหาแล้วไม่พบ	`find_by_guest` / `find_by_room` คืน None; เมนูพิมพ์ข้อความว่าไม่พบ	test_two_way_lookup
ยังไม่มีแขก (K = 0)	เพิ่ม/ลบอาคารรายงานย้าย 0 และ moved_rate = None พร้อมเหตุผล; `load_statistics` ให้ max = min = 0, mean = SD = 0 และ cv = None (ไม่หารด้วยศูนย์)	test_no_guests_rate_not_computed, test_empty_and_sorted_listing, test_load_statistics, test_load_report_with_no_guests

นอกจากนี้ รหัสอาคารว่าง/ช่องว่างล้วน, V < 1, รหัสแขกไม่ใช่จำนวนเต็มบวก, count ติดลบ/ทศนิยม และจำนวนช่องทางไม่ถูกต้อง ถูกปฏิเสธพร้อมข้อความ (ตรวจใน `test_edge_cases_leave_everything_unchanged`, `test_invalid`, `test_add_guests_range`, `test_init_with_different_guest_count_per_channel`)

4. ฟังก์ชันของโปรแกรม

รัน `python menu.py` มีเมนู 13 รายการ ต้องเลือกเมนู 1 (Run System) ก่อน เมนูแต่ละรายการเรียกเมธอดในตรรกะหลัก และมี try/except ครอบกันโปรแกรมหยุดทำงาน

โจทย์หัวข้อ 5	เมนู	เมธอดที่ทำงานจริง
1) เริ่มระบบด้วย N, V, จำนวนช่องทาง, จำนวนแขกต่อช่องทาง (s เริ่มที่ 1)	1. Run System	`HashRing(...)` + `GuestManager.populate_initial(channels, per_channel)` (เมนูรับเลขตัวเดียว หรือใส่ -1 เพื่อกรอกจำนวนทีละช่องทาง); รหัสอาคารสร้างเป็น NODE01, NODE02, ...
2) เพิ่มแขกรายบุคคล/กลุ่ม (ช่องทางเดิมและใหม่)	2. Add Guest; 3. Add Group	`add_guest`, `add_guests_range` ⟶ `add_guests` (all-or-nothing)

โจทย์หัวข้อ 5	เมนู	เมธอดที่ทำงานจริง
3) นำแขกออกด้วยรหัส (c, s)	4. Remove Guest	`remove_guest`
4) เพิ่ม/ลบอาคาร พร้อมจำนวน อัตราส่วน และรายการย้าย	5. Add Building; 6. Remove Building	`BuildingManager.add_building` / `remove_building` ⟶ `BuildingChangeResult` (พิมพ์จำนวน อัตรา ค่าคาดหวัง เวลา 3 ขั้นตอน และรายการย้าย 20 รายการแรก)
5) ค้นหาสองทาง	7. Find Guest by ID; 8. Find Room by ID	`find_by_guest(c, s)`; `find_by_room(node_id, room_no)`; ไม่พบแจ้งชัดเจน
6) แสดงห้องที่มีผู้พัก เรียงตาม node_id แล้ว room_no	9. Room Lists	`list_rooms()` (แสดง 100 รายการแรก พร้อมจำนวนทั้งหมด)
7) รายงานความสมดุลของโหลด	10. Load Reports	`load_per_node()` (รวมอาคาร 0 คน) + `load_statistics` (max, min, mean, SD, CV)
8) วัดเวลาและหน่วยความจำ พร้อมพารามิเตอร์	13. Measure Performance; experiments.py	`time.perf_counter`, `tracemalloc` (`measure_memory`) ตามหัวข้อ 6
9) เทียบ Consistent Hashing กับ hash mod N (ชุดแขกเดียวกัน)	12. Compare Consistent vs Modulo; experiments.py	`ModuloNodeSelector` + `BuildingManager` เดียวกัน (`do_compare_hashing`)
10) ส่งออก CSV UTF-8 (channel_id, sequence_id, node_id, room_no), รายการย้าย, ผลทดลอง	11. Export CSV; experiments.py	menu: results/menu_export_guests.csv และ menu_export_last_moves.csv; experiments.py: sample_guests.csv, experiment_B_moves.csv, experiment_*_rounds/summary.csv, experiment_C_loads.csv

5. การวิเคราะห์ Big-O

วิเคราะห์จากโค้ดที่ส่งจริง ใช้ N, V, M = N × V และ K ตามนิยามในหัวข้อ 1.1 และระบุทุกครั้งว่าเป็นค่าเฉลี่ย (average/expected) หรือกรณีเลวร้ายที่สุด (worst case)

5.1 ข้อสมมติฐาน

- ขนาดของรหัสแขก รหัสอาคาร ค่าแฮช และการเปรียบเทียบสองรหัสถือว่าคงที่ O(1) (fixed-size) ในความเป็นจริง Python ใช้จำนวนเต็มขยายขนาดได้ ถ้ารหัส c, s ยาวขึ้น (L หลัก) ต้นทุนการคำนวณ Cantor (การคูณ) และการแฮชข้อความ `guest:{c}:{s}` จะโตตาม L ไม่ใช่ O(1) อีกต่อไป การวิเคราะห์ด้านล่างจึงใช้ได้เมื่อ L ไม่ใหญ่มาก
- การดำเนินการของ `dict/set` เฉลี่ย O(1) กรณีเลวร้ายสุดที่คีย์ชนกันหมด O(K) ต่อครั้ง

- `bisect_left` / `insort` ทำ O(log M) ครั้งเปรียบเทียบ ส่วน `list.insert` เลื่อนสมาชิกท้าย list O(M) ต่อครั้ง
- ΔK คือจำนวนแขกที่ย้าย ค่าคาดหวังประมาณ $K/(N + 1)$ เมื่อเพิ่มอาคาร และ K/N เมื่อลบอาคาร (กรณีเลวร้ายสุด $\Delta K \le K$)
- ต้นทุนการรักษาความสอดคล้องของ `guests` และ `room_index` ถูกรวมไว้ในทุกแถวที่แก้ข้อมูล (เขียน dict 2 ชุดต่อแขก 1 คน คงที่เฉลี่ย O(1))

5.2 ตารางวิเคราะห์ตามขั้นตอนของโจทย์หัวข้อ 8

ขั้นตอน / เมธอด	เวลา เฉลี่ย	เวลา กรณีเลวร้าย	พื้นที่เสริม	ที่มาจากโค้ด
1ก) สร้างวงแหวนเริ่มต้น `HashRing.__init__` ⟶ `_build_initial_ring`	O(M log M)	O(M log M)	O(M)	ตรวจรหัสซ้ำ N รายการ (set) แฮช M จุด และ append แล้ว `sort()` ครั้งเดียว (Timsort) พื้นที่: list M ทูเพิล + ตัวช่วยเรียง
1ข) จัดแขกเริ่มต้น K คน `populate_initial` ⟶ `add_guests` ⟶ `_insert`	O(K log M)	O(K(K + log M))	O(K)	สร้างรายการรหัสและ set ตรวจซ้ำ O(K) แล้ว `_insert` K ครั้ง: ค้นวงแหวน O(log M) + Cantor O(1) + เขียน `guests` และ `room_index`. กรณีเลวร้าย dict ชนกัน O(K) ต่อครั้ง
2ก) เพิ่มแขก 1 คน `add_guest`	O(log M)	O(K + log M)	O(1)	ตรวจซ้ำใน dict + `get_node` + เขียน dict 2 ชุด
2ข) เพิ่มแขกกลุ่ม B คน `add_guests` / `add_guests_range`	O(B log M)	O(B(K + B + log M))	O(B)	ตรวจทั้งกลุ่มก่อน (set `seen` ขนาด B, ค้น dict) แล้วเพิ่มทีละคน; range สร้างรายการรหัส O(B)
2ค) นำแขกออก 1 คน `remove_guest`	O(1)	O(K)	O(1)	`get` + `del` บน dict 2 ชุด
3ก) ค้นด้วยรหัสแขก `find_by_guest`	O(log M)	O(K + log M)	O(1)	`guests.get` + คำนวณอาคารสด `ring.get_node` (แฮช + bisect) ไม่ใช้ผลที่บันทึกไว้
3ข) ค้นด้วยห้อง `find_by_room`	O(1)	O(K)	O(1)	`room_index.get` ครั้งเดียว
3ค) คำนวณอาคารจากวงแหวน `get_node`	O(log M)	O(log M)	O(1)	แฮช SHA-256 O(1) + `bisect_left`
4) เรียงรายการห้อง `list_rooms` (รวมสร้างรายการผลลัพธ์)	O(K log K)	O(K log K)	O(K)	สร้างทูเพิล (node_id, room_no, c, s) K ตัว แล้ว `sorted` เมนูพิมพ์เพียง 100 แถวแต่ยังเรียงครบ; ส่งออก CSV = list_rooms + เขียน K แถว
5ก) ปรับวงแหวนตอนเพิ่มอาคาร `add_node`	$O(V \cdot M) = O(V^2N)$	$O(V \cdot M)$	O(V)	แฮช V จุด แล้ว `bisect.insort` V ครั้ง: ค้น O(log M) + เลื่อน list O(M) ต่อจุด
5ก) ปรับวงแหวนตอนลบอาคาร `remove_node`	O(M)	O(M)	O(M)	สร้าง list ใหม่โดยกรองจุดของอาคารนั้นรอบเดียว (list เก่าถูกทิ้งหลังแทนที่)

ขั้นตอน / เมธอด	เวลา เฉลี่ย	เวลา กรณีเลวร้าย	พื้นที่เสริม	ที่มาจากโค้ด
5ข) หาแขกที่ได้รับผลกระทบ `find_misplaced` (แบบ ก) + สร้าง/เรียง `moves`	$O(K \log M + \Delta K \log \Delta K)$	$O(K \log M + \Delta K \cdot K + \Delta K \log \Delta K)$	$O(\Delta K)$	วนแขก K คน × `get_node` O(log M) ไม่ว่าจะมีผู้ย้ายกี่คน; จากนั้นสร้าง MoveRecord (ค้น dict ต่อคน), เรียง `moves` ตาม (c, s) และตรวจจติกา $O(\Delta K)$
5ค) ปรับข้อมูลและดัชนีของแขกที่ย้าย `move_guest` × ΔK	$O(\Delta K)$	$O(\Delta K \cdot K)$	O(1)	ต่อคน: get + del + set บน dict 2 ชุด (`room_no` ไม่เปลี่ยน)
5ค+) ตรวจความสอดคล้อง `verify_consistency` (เรียกทุกครั้งที่เพิ่ม/ลบ)	O(K log M)	O(K(K + log M))	O(1)	วนแขกทั้ง K คน: ค้น `room_index` และเรียก `get_node` ซ้ำ ไม่ถูกจับเวลาใน t_update_data

รวมต้นทุนเพิ่ม/ลบอาคาร 1 ครั้ง (`BuildingManager._change`): เพิ่มอาคาร เฉลี่ย $O(V \cdot M + K \log M + \Delta K \log \Delta K)$ ลบอาคาร เฉลี่ย $O(M + K \log M + \Delta K \log \Delta K)$ โดยทั่วไปเทอม $K \log M$ ครอบงำเพราะต้องไล่แขกทั้งหมดสองรอบ (หาผู้ย้าย และ verify) ส่วนจำนวนที่ถูกย้ายจริง ΔK เป็นเพียง “จำนวนข้อมูลที่เปลี่ยนตำแหน่ง” ซึ่งแยกจากเวลาประมวลผลทั้งหมด และอาจเป็น 0 ในชุดข้อมูลเล็ก

ข้อสังเกตเรื่องการจับเวลา: ค่า `t_update_data` (ขั้นตอน 5ค) จับเวลาเฉพาะลูป `move_guest` ไม่รวม `verify_consistency()` ซึ่งทำงานทุกครั้งหลังเปลี่ยนอาคารและมีต้นทุน $O(K \log M)$ จึงไม่ถูกนับในเวลาที่รายงานทั้ง 3 ขั้นตอน (ring adjust / find affected / update data) ใน results/experiment_B_*.csv

5.3 พื้นที่หน่วยความจำ (ขั้นตอนที่ 6)

ส่วน	พื้นที่ถาวร	พื้นที่ชั่วคราว	หมายเหตุ
วงแหวน (HashRing)	O(M) = O(NV) สำหรับ list + O(N) สำหรับ set รหัสอาคาร	สร้างวงแหวน: O(M) (ตัวช่วยเรียง); ลบอาคาร: list ใหม่ O(M); เพิ่มอาคาร: O(V)	ไม่ขึ้นกับ K
แขก (`guests`)	O(K)	-	Guest 1 วัตถุต่อแขก
ดัชนีเพิ่มเติม (`room_index`)	O(K)	-	ต้นทุนการค้นย้อน O(1)
รวมข้อมูลหลัก	O(NV + K)	-	วัด tracemalloc ในหัวข้อ 7
รายการห้อง `list_rooms`	-	O(K)	ใช้ตอนแสดง/ส่งออก
รายการย้าย (`misplaced`, `moves`)	เก็บใน BuildingChangeResult จนวัตถุผลลัพธ์ถูกปล่อย	$O(\Delta K)$ คาดหวัง K/N; เลวร้ายสุด O(K)	$\Delta K \le K$
เพิ่มแขกเป็นกลุ่ม/จัดแขกเริ่มต้น	-	O(B) หรือ O(K) (รายการรหัสทั้งหมดและ set ตรวจซ้ำอยู่ในหน่วยความจำพร้อมกัน)	ข้อจำกัดของกลุ่มขนาดใหญ่มาก

5.4 ตัวเลือกอาคารแบบ hash mod N (ใช้เทียบ)

การดำเนินการ	เวลาเฉลี่ย	เหตุผลจากโค้ด `ModuloNodeSelector`
`get_node`	O(1)	แฮช + `% len(nodes)` + ดัชนี list (ไม่ต้องค้นหา)
`add_node`	O(1)	`list.append` + `set.add` (เฉลี่ย)
`remove_node`	O(N)	`list.remove` ต้องไล่หาและเลื่อนสมาชิก
หาแขกที่ต้องย้าย	O(K)	`find_misplaced` เดิม แต่ `get_node` ของ mod เป็น O(1) ต่อคน
พื้นที่จัดเส้นทาง	O(N)	เก็บ list และ set ของรหัสอาคาร N รายการ จึง **ไม่ใช่ O(1)** (O(1) เป็นเพียงพื้นที่เพิ่มของสูตรหารเอาเศษ)

6. วิธีทดลอง

6.1 พารามิเตอร์ของชุดการทดลอง (หัวข้อ 7.1)

การทดลอง	ตัวแปรที่เปลี่ยน	ค่าที่กำหนด	ผลที่วัด
A ขนาดข้อมูล	K = 1,000; 10,000; 100,000	N = 10, V = 32	เวลาสร้างวงแหวน จัดแขก ค้นหา เรียงลำดับ และหน่วยความจำ
B จำนวนอาคาร	N = 5; 10; 20	K = 10,000, V = 32	เพิ่ม 1 อาคารและลบ 1 อาคาร: จำนวนย้าย อัตราย้าย เวลา 3 ขั้นตอน เทียบ hash mod N
C จำนวนจุดเสมือน	V = 1; 8; 32	N = 10, K = 10,000	แขกต่ออาคาร ความสมดุล (CV) หน่วยความจำ และการย้ายเมื่อเพิ่ม 1 อาคาร

- แขกสร้างจาก 10 ช่องทาง จำนวนเท่ากันทุกช่องทาง (K/10 คนต่อช่องทาง s = 1 ถึง K/10) รหัสอาคาร NODE01, NODE02, ... (NODE01 ยังเป็น NODE01 ไม่ว่า N จะเป็นเท่าใด)
- การทดลอง B: ลบ NODE01 เสมอ และเพิ่มอาคารชื่อ NODE_NEW เสมอ สร้างระบบใหม่ทุกครั้ง เพื่อให้ “เพิ่ม” และ “ลบ” เริ่มจากสถานะเดียวกัน; แถว hash mod N ใน CSV มี V = 0 หมายถึง “ไม่เกี่ยวข้อง”
- hash mod N (`ModuloNodeSelector`): เก็บรายการรหัสอาคารตามลำดับที่เพิ่ม เลือกอาคารที่ดัชนี H(guest) mod N; เพิ่มให้ต่อท้าย ลบให้เอารหัสนั้นออกโดยรักษาลำดับที่เหลือ; ใช้ฟังก์ชันแฮช รหัสแขก salt และชุดแขกเดียวกับ Consistent Hashing; เทียบการย้ายด้วยรหัสอาคารจริง (node_id ก่อน-หลัง) ไม่ใช่เทียบเพียงดัชนี

6.2 การทำซ้ำและ salt

ทุกการทดลองรัน 3 รอบ ใช้ salt = 0, 1, 2 (salt คือข้อความสั้นๆ ที่ต่อหน้าสตริงก่อนแฮช เช่น `salt:1:guest:3:7` เพื่อให้ตำแหน่งบนวงแหวนเป็นชุดใหม่ ทั้งที่ข้อมูลแขกและอาคารยังเป็นชุดเดิม) ทั้งสองวิธีในรอบเดียวกันใช้ salt เดียวกัน รายงานค่ามัธยฐานของ 3 รอบพร้อมช่วงต่ำสุด–สูงสุด (รูปแบบ มัธยฐาน (ต่ำสุด–สูงสุด))

จำนวนย้าย โหลด และตำแหน่งบนวงแหวนขึ้นกับค่าแฮชเท่านั้น จึงได้ผลเท่าเดิมทุกครั้งที่รัน ส่วนเวลาและหน่วยความจำขึ้นกับเครื่องและการรันแต่ละครั้ง

คำสั่งทำซ้ำ: **`python experiments.py`** (ต้องติดตั้ง matplotlib จาก requirements.txt เพื่อวาดกราฟ) จะเขียนทับ results/*.csv และ figures/*.png

6.3 วิธีวัด

- **เวลา:** **`time.perf_counter()`** แยกขั้นตอน: สร้างวงแหวน, จัดแขก K คน, เรียงรายการห้อง, และเพิ่ม/ลบอาคาร 3 ขั้นตอน (ปรับวงแหวน / หาแขกที่ต้องย้าย / ย้ายแขกและปรับดัชนี) ไม่รวมการพิมพ์ การรับแป้นพิมพ์ และการเขียนไฟล์
- **การค้นหา:** วัดเป็นชุด 1,000 ครั้ง (เลือกรหัสแขก 1,000 รหัสกระจายทั่วชุด; ถ้า K < 1,000 วนซ้ำจนครบ) แล้วหารเป็นต่อครั้ง หน่วยไมโครวินาที (µs) ได้แก่ **`ring.get_node`** อย่างเดียว, **`find_by_guest`**, **`find_by_room`**
- **หน่วยความจำ:** **`tracemalloc`** วัดส่วนที่เพิ่มจากจุดเริ่มต้นเดียวกัน รายงาน current และ peak เป็น MiB ใน “รอบวัดแยก” ต่างหากจากรอบจับเวลา (ไม่ให้ tracemalloc ทำให้เวลาช้าลง) เป็นหน่วยความจำที่ Python ติดตาม ไม่ใช่หน่วยความจำของทั้งโปรเซส (RSS)
- **ความสมดุล:** โหลดต่ออาคารนับรวมอาคารที่มี 0 คน; ค่าเฉลี่ย = K/N; SD ใช้ส่วนเบี่ยงเบนมาตรฐานประชากร (**`statistics.pstdev`**); CV = SD / ค่าเฉลี่ย (ยิ่งต่ำยิ่งสมดุล) ถ้า K = 0 แสดง max/min/mean/SD เป็น 0 และไม่คำนวณ CV
- ทุกรอบมีการตรวจด้วย assert/คอลัมน์: จำนวนย้ายที่รายงาน = จำนวนรายการย้าย, จำนวนแขกรวมคงเดิม, **`verify_consistency()`** เป็น True, และ rule_violations

6.4 เครื่องและเวอร์ชันที่ใช้

รายการ	ค่า
เครื่องที่สร้างไฟล์ results/*.csv และ figures/*.png (ไม่ได้บันทึกไว้ในไฟล์ CSV)	CPU: Intel Core i7-13620H (10C/16T) RAM: 16 GB OS: Windows 11 Home Single Language (build 26300) วันที่รัน: 18 ตุลาคม 2569
เวอร์ชัน Python	Python 3.13.5 (README ระบุว่าต้องการ 3.10 ขึ้นไป)
ไลบรารี	เฉพาะ Standard Library ในตรรกะหลัก (**`bisect`**, **`hashlib`**, **`statistics`**, **`time`**, **`tracemalloc`**, **`csv`**, **`dataclasses`**, **`unittest`**); matplotlib ≥ 3.8 เฉพาะ plots.py
การวัดเพิ่มเติมในรายงานนี้ (ค่า hash mod N ที่ CSV ไม่มี: เวลาค้นหา, CV, หน่วยความจำ)	รันด้วย report/extra_measurements.py บนเครื่องของสมาชิกอีกคน: CPU Intel Core i5-10300H, RAM 16 GB, Windows 11 Home, Python 3.13.14 โดยใช้คลาสของโปรเจกต์และพารามิเตอร์เดียวกับ A/C (N = 10, V = 32, K = 10,000, salt 0, 1, 2); ซึ่งเป็นคนละเครื่องกับที่สร้าง CSV จึงไม่ควรเทียบเวลากับค่าในไฟล์ผลทดลองโดยตรง

7. ผลการทดลอง

ตารางทุกตารางสร้างจาก results/*.csv โดยตรง (ค่ามัธยฐานของ 3 รอบ และช่วงต่ำสุด–สูงสุดในวงเล็บ) หน่วยแปลงจากไฟล์เป็น ms หรือ µs เพื่ออ่านง่ายเท่านั้น

7.1 การทดลอง A: ขนาดข้อมูล K (N = 10, V = 32)

ตารางที่ 1 เวลา (ms) เทียบ K

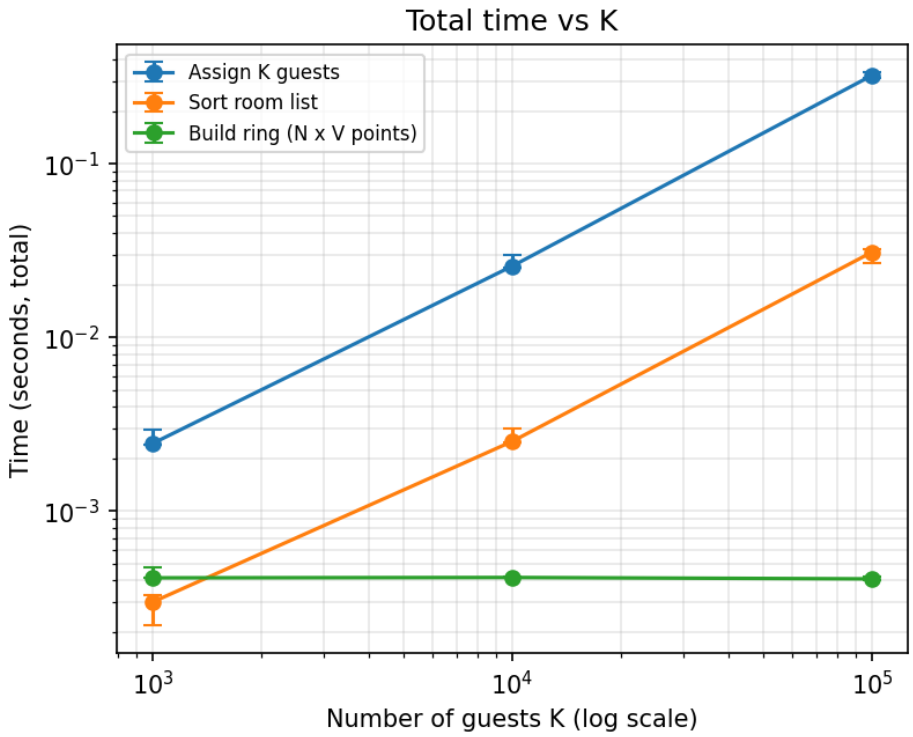
เวลา (ms)	K = 1,000	K = 10,000	K = 100,000
สร้างวงแหวน (M = 320 จุด)	0.414 (0.413–0.479)	0.416 (0.414–0.420)	0.408 (0.403–0.418)
จัดแขก K คน	2.455 (2.421–2.941)	25.785 (25.141–29.820)	322.820 (310.847–335.421)
เรียงรายการห้อง (list_rooms)	0.300 (0.221–0.327)	2.525 (2.476–3.003)	30.998 (26.788–32.261)

ตารางที่ 2 เวลาค้นหาต่อครั้ง (µs) จากชุด 1,000 ครั้ง

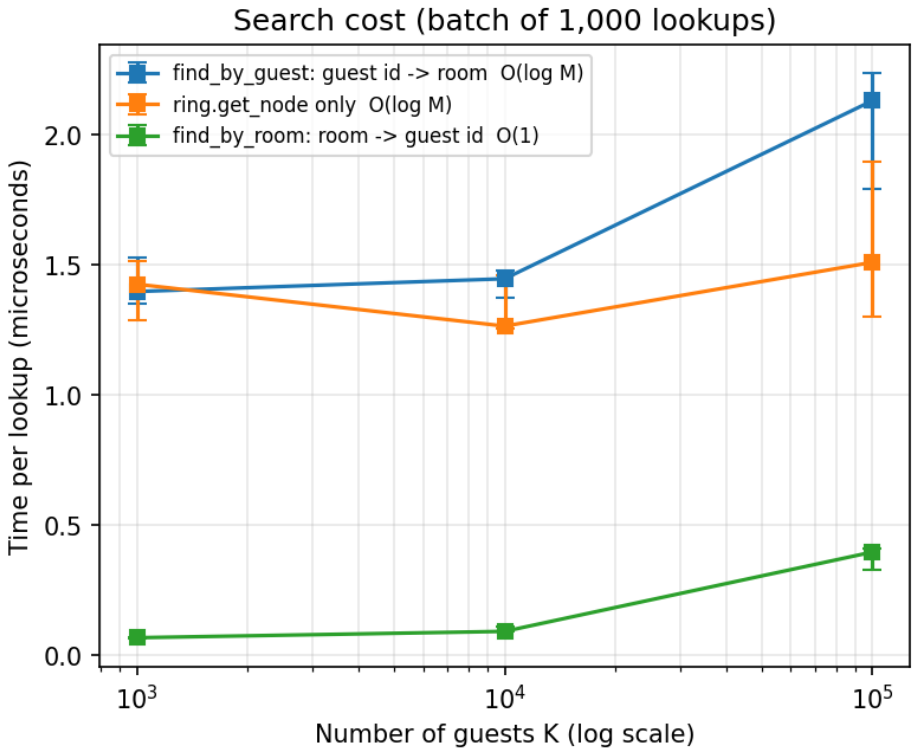
เวลาต่อครั้ง (µs)	K = 1,000	K = 10,000	K = 100,000
ring.get_node (คำนวณอาคาร)	1.426 (1.285–1.517)	1.265 (1.257–1.458)	1.510 (1.303–1.896)
find_by_guest (รหัสแขก ⟶ ห้อง)	1.397 (1.350–1.526)	1.447 (1.376–1.478)	2.131 (1.792–2.236)
find_by_room (ห้อง ⟶ รหัสแขก)	0.068 (0.065–0.068)	0.092 (0.092–0.110)	0.397 (0.330–0.412)

ตารางที่ 3 หน่วยความจำที่ Python ติดตาม (MiB) วงแหวน + แขก + ดัชนี

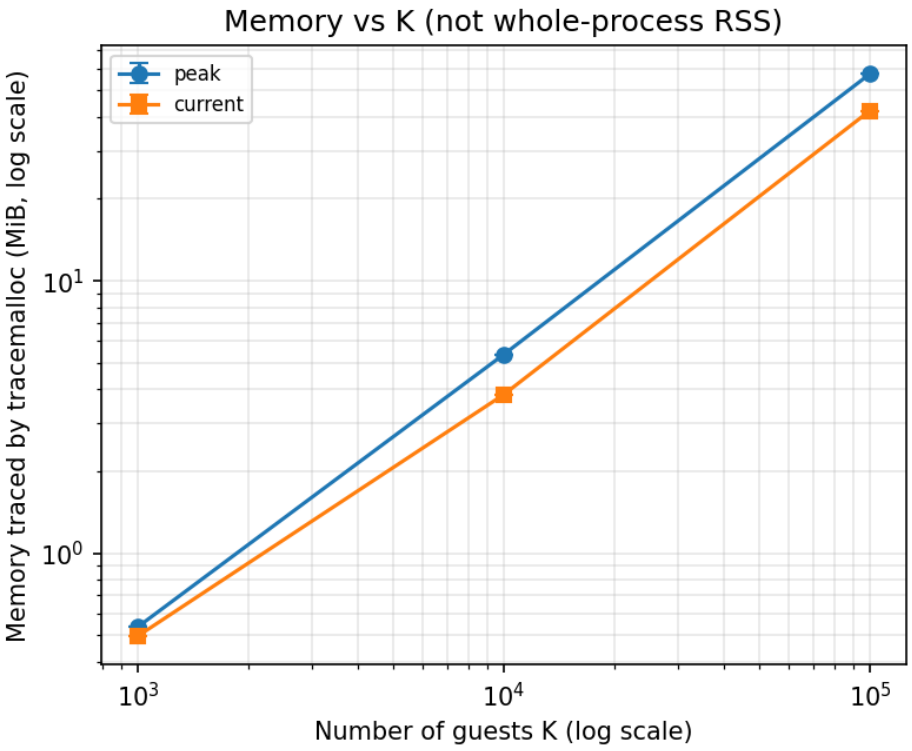
MiB	K = 1,000	K = 10,000	K = 100,000
current	0.497 (0.497–0.497)	3.825 (3.825–3.825)	42.101 (42.101–42.101)
peak	0.537 (0.537–0.537)	5.368 (5.368–5.368)	57.440 (57.440–57.440)
current ต่อแขก 1 คน (ไบต์ = current × 1,048,576 / K)	521	401	441



ภาพที่ 2(ก) เวลารวมของการจัดแขก K คน, การเรียงรายการห้อง และการสร้างวงแหวน เทียบ K (แกนตั้ง: วินาที, สเกลลอการิทึม; N = 10, V = 32, 10 ช่องทาง; มัธยฐาน 3 รอบ salt 0, 1, 2; แท่งความคลาดเคลื่อน = ต่ำสุด–สูงสุด) จาก figures/graph_A_time_vs_K.png



ภาพที่ 2(ข) เวลาค้นหาต่อครั้ง (µs) จากชุด 1,000 ครั้ง เทียบ K (พารามิเตอร์และแท่งความคลาดเคลื่อนเหมือนภาพที่ 2(ก))



ภาพที่ 2(ค) หน่วยความจำที่ tracemalloc ติดตาม (MiB, สเกลลอการิทึม; ไม่ใช่ RSS ของทั้งโปรเซส) เทียบ K (พารามิเตอร์เหมือนภาพที่ 2(ก))

การตีความ:

- เวลาสร้างวงแหวนแทบคงที่ที่ราว 0.416 ms ทุกค่า K เพราะขึ้นกับ M = 320 เท่านั้น (O(M log M)) ไม่ขึ้นกับ K
- เวลาจัดแขกโตตาม K: มัธยฐาน 2.455 ⟶ 25.785 ⟶ 322.820 ms (K เพิ่ม 10 เท่า เวลาเพิ่ม 10.50 และ 12.52 เท่า) สอดคล้องกับ O(K log M) เมื่อ M คงที่ โดยเวลาต่อแขกเพิ่มขึ้นเล็กน้อยที่ K = 100,000 (2.58 µs ⟶ 3.23 µs ต่อคน) ซึ่งเราคาดว่ามาจากข้อมูลที่ใหญ่ขึ้นจนใช้แคชของ CPU และการขยายตาราง dict มากขึ้น (เป็นข้อสันนิษฐาน ไม่ได้วัดแยก)
- เวลาเรียงรายการห้องโตเร็วกว่า K เล็กน้อย (0.300 ⟶ 2.525 ⟶ 30.998 ms; เพิ่ม 8.42 และ 12.28 เท่า) สอดคล้องกับ O(K log K)
- การค้นด้วยรหัสแขกอยู่ราว 1.4–2.1 µs และการคำนวณอาคารอย่างเดียว (get_node) ราว 1.3–1.5 µs ซึ่งแทบไม่ขึ้นกับ K ตามที่คาด (O(log M) บน M = 320 คงที่ และเวลาส่วนใหญ่เป็นต้นทุนแฮช SHA-256 กับการสร้างสตริง) ส่วนการค้นด้วยห้องเร็วที่สุด (0.068 µs ที่ K = 1,000) แต่ **ไม่คงที่เป๊ะ**: เพิ่มเป็น 0.397 µs ที่ K = 100,000 แม้ทฤษฎีเฉลี่ยเป็น O(1) เราคาดว่าเป็นผลของแคชเมื่อ dict ใหญ่ขึ้น (สันนิษฐาน) และค่า find_by_guest ที่ K = 100,000 (2.131 µs) มีช่วงต่ำสุด–สูงสุดกว้างกว่าที่ K อื่น

- หน่วยความจำ current โต 0.497 ⟶ 3.825 ⟶ 42.101 MiB หรือราว 401–441 ไบต์ต่อแขก (Guest + ทูเพิลคีย์ + 2 dict) สอดคล้องกับ O(NV + K) ที่เทอม K ครอบงำ; peak สูงกว่า current ราว 1.1–1.4 เท่า (ยิ่ง K มากยิ่งต่างมาก) เพราะมีรายการรหัสและ set ตรวจซ้ำชั่วคราวขณะจัดแขก

7.2 การทดลอง B: จำนวนอาคาร N (K = 10,000, V = 32)

ตารางที่ 4 จำนวนแขกที่ย้ายเมื่อเพิ่ม 1 อาคาร (NODE_NEW) ค่าคาดหวังตามโจทย์ K/(N+1) ใช้กับ Consistent Hashing

N	ค่าคาดหวัง K/(N+1)	Consistent Hashing: ย้าย (คน) และอัตรา	hash mod N: ย้าย (คน) และอัตรา
5	1,666.7	1,859 (1,363–2,031) 18.59% (13.63%–20.31%)	8,337 (8,290–8,381) 83.37% (82.90%–83.81%)
10	909.1	1,068 (806–1,080) 10.68% (8.06%–10.80%)	9,100 (9,069–9,101) 91.00% (90.69%–91.01%)
20	476.2	374 (368–571) 3.74% (3.68%–5.71%)	9,520 (9,513–9,529) 95.20% (95.13%–95.29%)

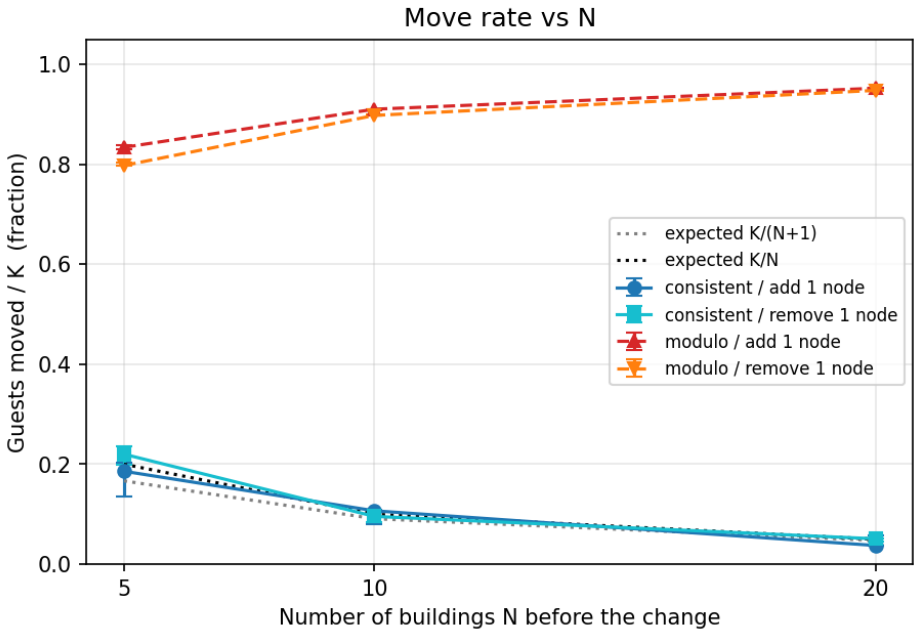
ตารางที่ 5 จำนวนแขกที่ย้ายเมื่อลบ 1 อาคาร (NODE01) ค่าคาดหวัง K/N

N	ค่าคาดหวัง K/N	Consistent Hashing: ย้าย (คน) และอัตรา	hash mod N: ย้าย (คน) และอัตรา
5	2,000.0	2,200 (1,983–2,352) 22.00% (19.83%–23.52%)	7,974 (7,969–8,027) 79.74% (79.69%–80.27%)
10	1,000.0	956 (843–1,052) 9.56% (8.43%–10.52%)	8,976 (8,966–9,005) 89.76% (89.66%–90.05%)
20	500.0	508 (455–522) 5.08% (4.55%–5.22%)	9,476 (9,470–9,495) 94.76% (94.70%–94.95%)

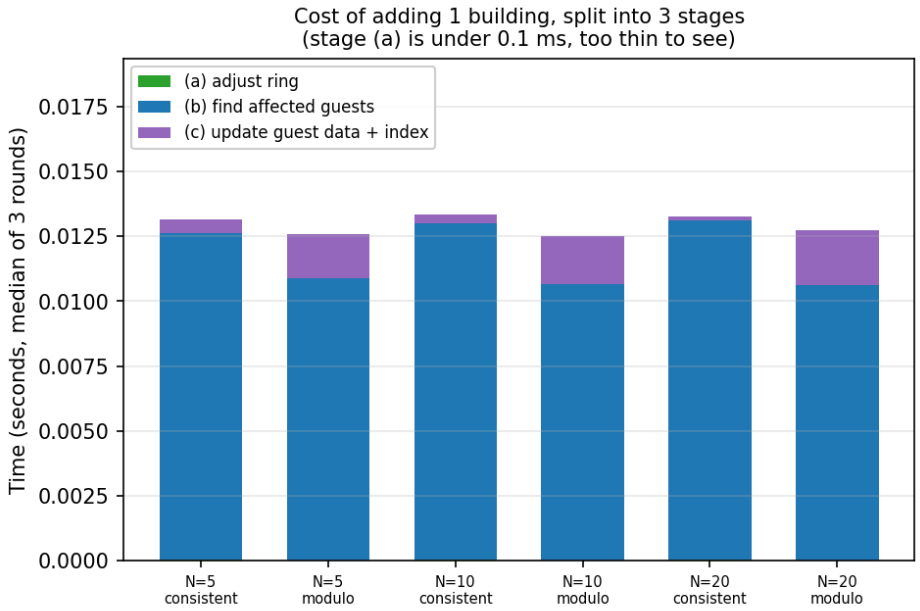
ตารางที่ 6 เวลาเพิ่ม/ลบอาคารแยก 3 ขั้นตอน (มัธยฐาน; ring adjust เป็น µs, find affected และ update data เป็น ms; ไม่รวม verify_consistency)

N	วิธี	ดำเนินการ	(ก) ปรับวงแหวน (µs)	(ข) หาแขกที่ย้าย (ms)	(ค) ย้ายแขก+ดัชนี (ms)
5	Consistent	เพิ่ม	51.7	12.575	0.520
5	Consistent	ลบ	15.2	12.609	0.569
5	hash mod N	เพิ่ม	2.7	10.893	1.694
5	hash mod N	ลบ	7.5	10.840	1.775
10	Consistent	เพิ่ม	49.7	12.954	0.352
10	Consistent	ลบ	21.8	12.838	0.343
10	hash mod N	เพิ่ม	2.0	10.652	1.837
10	hash mod N	ลบ	3.3	10.508	2.126

N	วิธี	ดำเนินการ	(ก) ปรับวงแหวน (µs)	(ข) หาแขกที่ย้าย (ms)	(ค) ย้ายแขก+ดัชนี (ms)
20	Consistent	เพิ่ม	51.1	13.047	0.169
20	Consistent	ลบ	30.3	12.807	0.135
20	hash mod N	เพิ่ม	2.0	10.616	2.105
20	hash mod N	ลบ	7.0	11.170	2.023



ภาพที่ 3(ก) อัตราการย้าย (ย้าย/K) เทียบ N ของ Consistent Hashing และ hash mod N เมื่อเพิ่มและลบ 1 อาคาร พร้อมเส้นค่าคาดหวัง K/(N+1) และ K/N (แกนตั้ง: สัดส่วนของ K ไม่มีหน่วย; K = 10,000, V = 32; มัธยฐาน 3 รอบ salt 0, 1, 2; แท่ง = ต่ำสุด–สูงสุด) จาก figures/graph_B_move_rate_vs_N.png



ภาพที่ 3(ข) เวลาเพิ่ม 1 อาคารแยก 3 ขั้นตอน (วินาที, มัธยฐาน 3 รอบ; พารามิเตอร์เหมือนภาพที่ 3(ก)) ขั้นตอน (ก) น้อยกว่า 0.1 ms จึงบางจนมองไม่เห็นในกราฟ ดูตัวเลขในตารางที่ 6

การตีความ:

- Consistent Hashing ย้ายน้อยกว่า hash mod N มาก และอัตราย้ายลดลงเมื่อ N เพิ่ม: เมื่อเพิ่ม 1 อาคาร มัธยฐาน 18.59%, 10.68%, 3.74% สำหรับ N = 5, 10, 20 ขณะที่ hash mod N ย้าย 83.37%, 91.00%, 95.20% ซึ่งใกล้ค่า N/(N + 1) ที่ได้จากการวิเคราะห์ (83.33%, 90.91%, 95.24%): คู่ (h mod N, h mod (N + 1)) เท่ากันเพียง 1 ใน N + 1 ของค่า h จึงมีแขกอยู่ที่เดิมแค่ราว 1/(N + 1)
- เมื่อลบ NODE01 ซึ่งเป็นรหัสแรกของรายการ hash mod N ดัชนีของอาคารที่เหลือเลื่อนลงหมด จึงย้ายราว (N − 1)/N (80%, 90%, 95% จากการวิเคราะห์เทียบกับค่าวัด 79.74%, 89.76%, 94.76%) ส่วน K/N ของโจทย์เป็นค่าคาดหวังของ Consistent Hashing ไม่ใช่ของ hash mod N
- **กติกา 4.4 ข้อ 2:** ใน CSV ทั้ง 18 แถวของ Consistent Hashing มี rule_violations = 0 (ย้ายเข้าอาคารใหม่เท่านั้นเมื่อเพิ่ม และย้ายเฉพาะแขกของอาคารที่ลบเมื่อลบ) ส่วนทั้ง 18 แถวของ hash mod N มี rule_violations > 0 (เช่น N = 10, salt 0, เพิ่มอาคาร: ย้าย 9,100 คน ในจำนวนนี้ 8,249 คนไม่ได้ย้ายเข้าอาคารใหม่) แสดงว่า hash mod N ไม่ผ่านกติกาข้อนี้ แม้ข้อมูลยังสอดคล้อง (consistent = True ทุกแถว)
- **ค่าที่วัดต่างจาก K/(N+1) และ K/N อย่างไร:** เมื่อเพิ่มอาคาร มัธยฐานเทียบค่าคาดหวัง: N = 5: 1,859 เทียบ 1,666.7 (+11.5%), N = 10: 1,068 เทียบ 909.1 (+17.5%), N = 20: 374 เทียบ 476.2 (-21.5%); เมื่อลบอาคาร N = 5: 2,200 เทียบ 2,000 (+10.0%), N = 10: 956 เทียบ 1,000 (-4.4%), N = 20: 508 เทียบ 500 (+1.6%) ค่าคาดหวังทุกค่าอยู่ภายในช่วงต่ำสุด–สูงสุดของ 3 รอบ แต่มัธยฐานเบี่ยงได้ถึงราว 10–20% (เช่น เพิ่มอาคารที่ N = 5 มัธยฐานสูงกว่าคาด แต่ salt 2 ให้ 1,363 ต่ำกว่าคาด) สาเหตุ: (1) K/(N + 1) และ K/N เป็นค่าเฉลี่ยเมื่อช่วงบนวงแหวนที่อาคารหนึ่งครอบครองเท่ากับ 1/N พอดี แต่จริงๆ ตำแหน่งของ virtual node 32

จุดต่ออาคารเป็นค่าแฮช ทำให้ส่วนแบ่งของอาคารใหม่ (หรือของอาคารที่ลบ) แกว่งรอบ 1/N; ความแกว่งนี้วัดได้จากการทดลอง C: ที่ V = 32 ค่า CV ของโหลดคือ 0.144 (0.130–0.180) หรือราว 13–18% ใกล้เคียงกับขนาดส่วนเบี่ยงเบนข้างต้น; (2) V จำกัดที่ 32 (V มากขึ้นความแกว่งลดลงแต่ใช้หน่วยความจำมากขึ้น); (3) แต่ละ salt ใช้ชุดแขกเพียงชุดเดียว และมีเพียง 3 รอบ จึงบอกความแปรปรวนจริงได้เพียงคร่าวๆ

- เวลา: ขั้นตอน (ข) หาแขกที่ต้องย้ายครองเวลาเกือบทั้งหมด (ราว 13.0 ms ที่ K = 10,000 หรือ 1.30 µs ต่อแขก ใกล้เคียงเวลา get_node ในตารางที่ 2) ทั้งสองวิธีใช้เวลาหาแขกในระดับเดียวกัน เพราะทั้งคู่วนแขกครบ K คน (แบบ ก) hash mod N เร็วกว่าเล็กน้อย (ไม่ต้อง bisect) แต่ขั้นตอน (ค) ใช้เวลานานกว่าเพราะย้ายแขกมากกว่า 3.6–25.5 เท่า (ตัวอย่าง N = 10 เพิ่มอาคาร: 1.837 ms เทียบ 0.352 ms) ขั้นตอน (ก) ของ Consistent Hashing เพิ่มอาคารใช้ราว 50 µs ไม่เพิ่มตาม N ทั้งที่ O(V·M) คาดว่าโตตาม M; เพราะใน M ≤ 640 การเลื่อนสมาชิกใน list เร็วมาก เทอมที่ครอบงำคือแฮช 32 จุด (32 × ~1.3 µs ≈ 42 µs) ส่วนการลบ (O(M)) โตตาม N จริง: 15.2 ⟶ 21.8 ⟶ 30.3 µs

7.3 การทดลอง C: จำนวนจุดเสมือน V (N = 10, K = 10,000)

ตารางที่ 7 ผลของ V ต่อความสมดุลของโหลด การย้ายเมื่อเพิ่ม 1 อาคาร (ค่าคาดหวัง K/(N+1) = 909.1) และหน่วยความจำ

V	M = N×V	max โหลด	min โหลด	SD	CV	ย้ายเมื่อเพิ่ม 1 อาคาร
1	10	2,010 (1,638–3,471)	84 (8–245)	601.5 (448.1–1112.7)	0.6015 (0.4481–1.1127)	185 (78–582) 1.85% (0.78%–5.82%)
8	80	1,303 (1,297–1,518)	538 (376–571)	262.7 (260.1–313.3)	0.2627 (0.2601–0.3133)	637 (441–1,482) 6.37% (4.41%–14.82%)
32	320	1,242 (1,194–1,267)	732 (677–782)	143.8 (130.4–180.4)	0.1438 (0.1304–0.1804)	1,068 (806–1,080) 10.68% (8.06%–10.80%)

ค่าเฉลี่ยโหลด = 1,000 คนต่ออาคารทุกกรณี (K/N)

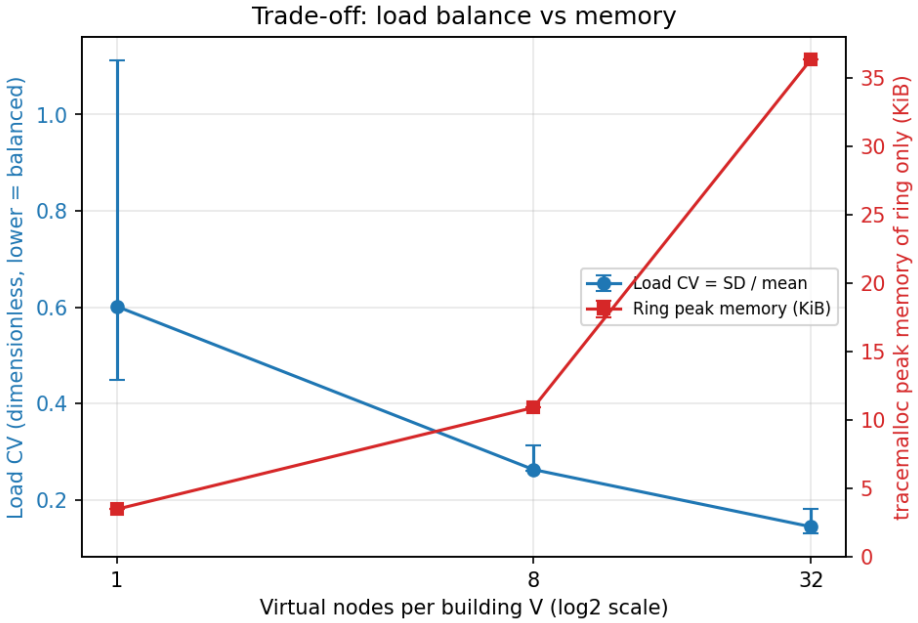
ตารางที่ 8 หน่วยความจำที่ tracemalloc ติดตาม (current / peak)

V	เฉพาะวงแหวน (KiB)	วงแหวน + แขก 10,000 คน (MiB)
1	2.3 / 3.5	3.793 / 5.335
8	9.7 / 10.9	3.800 / 5.343
32	35.2 / 36.4	3.825 / 5.368

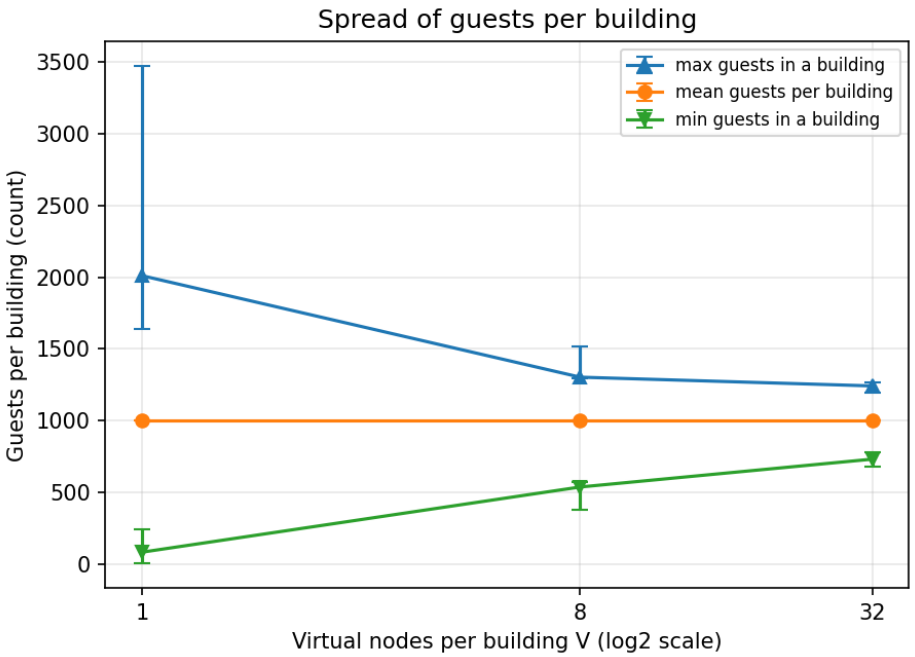
ตารางที่ 9 จำนวนแขกต่ออาคาร (จาก results/experiment_C_loads.csv, salt 0, N = 10, K = 10,000)

อาคาร	V = 1	V = 8	V = 32
NODE01	1,362	823	843
NODE02	766	1,268	1,066
NODE03	408	723	861
NODE04	84	1,297	1,092
NODE05	588	538	1,194
NODE06	2,010	1,240	1,156
NODE07	1,384	761	1,052
NODE08	385	931	782
NODE09	1,590	1,233	982
NODE10	1,423	1,186	972
รวม	10,000	10,000	10,000

ไฟล์เดียวกันมี salt 1 และ 2 ครบทุก V (รวม 90 แถว = 3 ค่า V × 3 salt × 10 อาคาร) ผลรวมของแต่ละชุดเท่ากับ 10,000



ภาพที่ 4(ก) แลกเปลี่ยนระหว่างความสมดุลของโหลด (CV = SD/ค่าเฉลี่ย ไม่มีหน่วย; ต่ำ = สมดุล) กับหน่วยความจำ peak ของวงแหวน (KiB) เมื่อ V เปลี่ยน (N = 10, K = 10,000; มัธยฐาน 3 รอบ salt 0, 1, 2; แท่ง = ต่ำสุด–สูงสุด; แกน V สเกล log2) จาก figures/graph_C_effect_of_V.png



ภาพที่ 4(ข) จำนวนแขกต่ออาคาร (คน): มากที่สุด ค่าเฉลี่ย และน้อยที่สุด เมื่อ V เปลี่ยน (พารามิเตอร์เหมือนภาพที่ 4(ก))

การตีความ:

- V มากขึ้น โหลดสมดุลขึ้นมาก: CV มัธยฐาน 0.6015 (V = 1) ⟶ 0.2627 (V = 8) ⟶ 0.1438 (V = 32) ที่ V = 1 อาคารที่รับมากที่สุดมีมัธยฐาน 2,010 คน (สูงสุดใน 3 รอบ 3,471) และน้อยที่สุด 84 คน (ต่ำสุดใน 3 รอบ 8) เทียบกับค่าเฉลี่ย 1,000
- ราคาที่จ่ายคือหน่วยความจำของวงแหวน: peak 3.5 ⟶ 10.9 ⟶ 36.4 KiB แต่เมื่อรวมข้อมูลแขก 10,000 คน แล้วต่างกันเพียง 0.032 MiB จาก 5.34 MiB เพราะเทอม K ครอบงำ
- จำนวนย้ายเมื่อเพิ่ม 1 อาคารโตตาม V: 185 ⟶ 637 ⟶ 1,068 คน (ค่าคาดหวัง 909.1) เพราะจำนวนที่ย้ายเท่ากับโหลดของอาคารใหม่ และเมื่อ V น้อย อาคารใหม่ได้ช่วงวงแหวนสั้นและแกว่งมาก (V = 1 ช่วงกว้าง 78–582 คน) ค่าคาดหวัง K/(N+1) จึงใช้ได้ดีเมื่อ V มากพอ; ที่ V = 32 ค่าอยู่ในช่วง 806–1,080 คน
- ตัวเลขนี้เป็นค่าจากข้อมูลชุดเดียวต่อ salt (3 ชุด) ไม่ใช่ความน่าจะเป็นที่ประมาณได้แม่นยำ ข้อสรุปคือแนวโน้ม ไม่ใช่ค่าเฉพาะ

8. ตารางเปรียบเทียบ hash mod N กับ Consistent Hashing

แยก “ค่าจากการวิเคราะห์” ออกจาก “ค่าที่วัดได้จริง” (หัวข้อ 8.2 ของโจทย์) ค่าที่วัดจริงของ Consistent Hashing มาจาก results/*.csv (N = 10, V = 32, K = 10,000 เว้นแต่ระบุ) ค่าของ hash mod N ที่ไม่มีใน CSV (เวลาค้นหา, CV, หน่วยความจำ) ผู้จัดทำวัดเพิ่มด้วยสคริปต์ report/extra_measurements.py โดยใช้คลาส `ModuloNodeSelector/GuestManager` ของโปรเจกต์ และพารามิเตอร์/salt ชุดเดียวกัน (3 รอบ มัธยฐาน (ต่ำสุด–สูงสุด))

ประเด็น	ประเภทค่า	hash mod N	Consistent Hashing
การคำนวณอาคาร	วิเคราะห์	O(1): แฮช + หารเอาเศษ + ดัชนี list	O(log M) = O(log NV): แฮช + bisect_left
	วัดจริง (µs ต่อครั้ง, ชุด 1,000)	1.83 (1.76–2.19) (วัดเพิ่ม)	1.265 (1.257–1.458) (CSV, K = 10,000) 2.21 (2.20–2.37) (วัดเพิ่ม ในรอบเดียวกับ hash mod N)
การเพิ่ม/ลบอาคาร (ปรับโครงสร้าง)	วิเคราะห์	เพิ่ม O(1), ลบ O(N); หาผู้ย้าย O(K); ย้าย ΔK	เพิ่ม O(V·M), ลบ O(M); หาผู้ย้าย (แบบ ก) O(K log M); ย้าย ΔK; verify O(K log M)
	วัดจริง (N = 10; ปรับวงแหวน µs / หาผู้ย้าย ms / ย้าย ms)	เพิ่ม 2.0 / 10.65 / 1.84 ลบ 3.3 / 10.51 / 2.13	เพิ่ม 49.7 / 12.95 / 0.35 ลบ 21.8 / 12.84 / 0.34
อัตราการย้าย (เพิ่ม 1 อาคาร)	วิเคราะห์	≈ N/(N+1) (N = 5, 10, 20 ⟶ 83.33%, 90.91%, 95.24%)	คาดหวัง ≈ 1/(N+1) (16.67%, 9.09%, 4.76%)
	วัดจริง (มัธยฐาน)	83.37% / 91.00% / 95.20%	18.59% / 10.68% / 3.74%
อัตราการย้าย (ลบ NODE01)	วิเคราะห์	≈ (N−1)/N เมื่อลบอาคารแรกของรายการ (80%, 90%, 95%)	คาดหวัง ≈ 1/N (20%, 10%, 5%)
	วัดจริง (มัธยฐาน)	79.74% / 89.76% / 94.76%	22.00% / 9.56% / 5.08%
พื้นที่จัดเส้นทาง (Routing space)	วิเคราะห์	O(N): list + set ของรหัสอาคาร	O(NV) = O(M): M ทูเพิลใน list (+ O(N) set)
	วัดจริง (tracemalloc current / peak, KiB; N = 10)	1.51 / 1.80 (วัดเพิ่ม)	35.2 / 36.4 (CSV, V = 32)
พื้นที่ข้อมูลรวม	วิเคราะห์	O(N + K)	O(NV + K)
	วัดจริง (MiB current / peak; K = 10,000)	3.792 / 5.335 (วัดเพิ่ม)	3.825 / 5.368 (CSV)

ประเด็น	ประเภทค่า	hash mod N	Consistent Hashing
ความสมดุลของโหลด	วิเคราะห์	แฮชกระจายสม่ำเสมอ: โหลดต่ออาคาร ≈ binomial จึง CV ต่ำ (ประมาณ $\sqrt{(N-1)/K} \approx 0.03$ ที่ N = 10, K = 10,000)	ขึ้นกับ V: V น้อยโหลดเอียง, V มากเข้าใกล้สมดุล (แต่ไม่ถึงเท่า mod)
	วัดจริง (CV, มัธยฐาน (ต่ำสุด–สูงสุด); N = 10, K = 10,000)	0.0332 (0.0325–0.0372) (วัดเพิ่ม)	V = 1: 0.6015 (0.4481–1.1127) V = 8: 0.2627 (0.2601–0.3133) V = 32: 0.1438 (0.1304–0.1804) (CSV)

อ่านผลอย่างระวัง: (1) เวลาค้นหาของสองวิธีใกล้กัน เพราะต้นทุนส่วนใหญ่คือการแฮช SHA-256 กับสร้างสตริง ไม่ใช่การค้นวงแหวน ในรอบวัดเพิ่มเดียวกัน Consistent Hashing ช้ากว่าราว 20% (ค่า 2 ค่าที่วัดต่างรอบ/ต่างเครื่องไม่ควรเอามาเทียบกันโดยตรง ตัวเลขของ CSV เร็วกว่าเพราะวัดคนละช่วงเวลา) (2) hash mod N สมดุลโหลดดีกว่ามาก (CV ราว 0.033 เทียบกับ 0.144 ของ V = 32) แต่แลกด้วยการย้ายแขกเกือบทั้งระบบเมื่อเปลี่ยนจำนวนอาคาร และผิดกติกา 4.4 ข้อ 2 (3) Consistent Hashing ใช้หน่วยความจำจัดเส้นทางมากกว่า (ราว 35 KiB เทียบกับ 1.5 KiB) แต่เมื่อรวมข้อมูลแขก 10,000 คนต่างกันเพียงราว 0.03 MiB (4) การวิเคราะห์ CV ของ hash mod N เป็นการประมาณแบบ binomial ตรวจกับค่าวัดได้ใกล้เคียง (0.033) แต่เป็นการประมาณ ไม่ใช่การพิสูจน์

9. การทดสอบความถูกต้อง

9.1 ตัวอย่างวงแหวนเล็ก (หัวข้อ 9.1)

ใช้วงแหวนขนาด 100 และ V = 1 โดยกำหนดตำแหน่งเอง (`add_node(node_id, custom_positions=[...])`) วางอาคาร A ที่ 20, B ที่ 50, C ที่ 80 แล้วค้นด้วยกติกา “มากกว่าหรือเท่ากับ” และวนกลับต้นวง `test_section_9_1_small_ring_example` คำนวณผลทั้ง 3 สถานะเทียบกับตารางของโจทย์ และนับจำนวนที่ต่าง:

ตำแหน่งแฮชของแขก	เริ่มต้น A B C	เพิ่ม D ที่ 35	ลบ B จากสถานะเริ่มต้น
10	A	A	A
20	A	A	A
25	B	D	C
35	B	D	C
50	B	B	C
70	C	C	C
90	A	A	A
จำนวนที่ย้ายจากสถานะเริ่มต้น	-	2 คน (25, 35)	3 คน (25, 35, 50)

ตารางนี้ตรงกับตารางในโจทย์หัวข้อ 9.1 ทุกช่อง (7 แถว) การทดสอบใช้ assert เทียบรายการ [A, A, B, B, B, C, A] ⟶ [A, A, D, D, B, C, A] ⟶ [A, A, C, C, C, C, A] และตรวจจำนวนย้าย 2 และ 3 ตำแหน่ง 90 เลย C ที่ 80 จึงวนกลับไปที่ A (จุดแรกของวงแหวน) ทุกสถานะ

9.2 ความครอบคลุมของการทดสอบตามโจทย์หัวข้อ 9.2 และ 4.5

หัวข้อ	สิ่งที่ต้องตรวจ	เทสที่ตรวจ (test_all.py)
9.2 ข้อ 1	เพิ่มแขกใหม่ทั้งช่องทางเดิมและใหม่ ที่อยู่ห้องแขกเดิมไม่เปลี่ยน	test_9_2_1_new_guests_never_move_existing_guests; test_room_unchanged_after_ring_change
9.2 ข้อ 2	เลขห้องตามสูตร และรหัสต่างกันไม่ได้ห้องซ้ำ	test_known_values; test_unique; test_invalid
9.2 ข้อ 3	ปฏิเสธรหัสซ้ำ รวมกลุ่มที่ซ้ำบางรายการ ข้อมูลเดิมไม่เปลี่ยน	test_9_2_3_rejected_batch_leaves_data_untouched; test_group_all_or_nothing; test_add_remove_and_duplicates; test_add_guests_range
9.2 ข้อ 4	ค้นที่ตำแหน่งตรงจุด ระหว่างจุด หลังจุดสุดท้าย (วนกลับ) และ virtual node ชนกัน	test_section_9_2_4_search_and_collision; test_section_9_1_small_ring_example
9.2 ข้อ 5	เพิ่ม/ลบอาคารเมื่อ V > 1 (V = 16) ผู้ย้ายตามกติกา จำนวนแขกไม่หาย ไม่เพิ่ม และจำนวนรายการย้ายตรงกับยอดที่รายงาน	test_section_4_4_2_ring_movement_rules; test_add_building_moves_only_to_new_building; test_remove_building_moves_only_its_guests; test_move_records_and_room_lookup; test_add_then_remove_restores_original
9.2 ข้อ 6	กรณีขอบ 4.5 ทั้งหมด และจำนวนแถว CSV = จำนวนแขกปัจจุบัน	ดูตาราง 3.5; test_9_2_6_csv_rows_match_current_guest_count; test_csv_export_format; test_menu_export_csv_action
4.5 K = 0	ค่าเฉลี่ยไม่หารศูนย์ CV และอัตราย้ายไม่คำนวณ	test_no_guests_rate_not_computed; test_load_statistics; test_load_report_with_no_guests; test_empty_and_sorted_listing
หัวข้อ 5 และ 7 (เพิ่มเติม)	เริ่มระบบด้วยจำนวนแขกต่อช่องทางต่างกัน, SHA-256 คงที่ข้ามการรัน, เมนูเปรียบเทียบและวัดผล	test_init_with_different_guest_count_per_channel; test_sha256_fixed_values; test_sha256_64bit_and_prefix_separation; test_menu_compare_and_performance; test_menu_input_handling_and_mock; test_expected_value_and_hash_ring_beats_modulo; test_timing_fields_present; test_two_way_lookup

9.3 ผลการรันชุดทดสอบ

รันด้วยคำสั่ง **`python -m unittest test_all -v`** (Python 3.13.5) ผลดิบบันทึกที่ results/test_report.txt: **`Ran 34 tests in 0.099s`** ผลสรุป **`OK`** ผ่านทั้งหมด 34 จาก 34 ข้อ ไม่มีข้อที่ไม่ผ่าน ไม่มีข้อที่ error หรือถูกข้าม

#	เทส	สิ่งที่ตรวจ (บรรทัดแรกของ docstring)	ผล
คลาส TestPart1_HashRingCore			
1	test_section_4_4_2_ring_movement_rules	เมื่อ V > 1 การเพิ่ม node ต้องย้ายเฉพาะคนที่ไป node ใหม่ และการลบต้องไม่กระทบคนอื่น	ผ่าน
2	test_section_4_5_node_edge_cases_and_modulo	กรณีขอบ (เพิ่มซ้ำ, ลบที่ไม่มี, ลบตัวสุดท้าย) และ ModuloNodeSelector	ผ่าน
3	test_section_9_1_small_ring_example	ลองตาม ring เล็กๆ ในโจทย์ 9.1	ผ่าน
4	test_section_9_2_4_search_and_collision	ลองค้นหาทั้งแบบตรงจุด อยู่ระหว่างจุด วนกลับต้นวง และ virtual node ที่ position ชนกัน	ผ่าน
5	test_sha256_64bit_and_prefix_separation	hash 64-bit ต้องได้ค่าเดิมทุกครั้ง และ guest กับ node ต้องไม่ปนกัน	ผ่าน
คลาส TestPart2_CantorRoomNo			
6	test_invalid	เช็คว่า c หรือ s ที่เป็น 0 หรือติดลบ ต้องโดนปฏิเสธด้วย ValueError (ไม่คำนวณต่อ)	ผ่าน
7	test_known_values	เช็คค่าที่คำนวณมือได้จากตารางในหัวข้อ 4.2: (1,1)->1, (2,1)->2, (1,2)->3, (3,1)->4, (2,2)->5	ผ่าน
8	test_unique	เช็คว่ารหัสแขกคนละคู่ต้องได้เลขห้องไม่ซ้ำกัน	ผ่าน
คลาส TestPart2_GuestManager			
9	test_add_guests_range	ทดสอบการเพิ่มแขกแบบระบุช่วง (c, s_start, count) ตามหัวข้อ 4.1 ข้อ 3	ผ่าน
10	test_add_remove_and_duplicates	กรณีขอบเรื่องแขก: เพิ่มรหัสซ้ำต้องถูกปฏิเสธ, ลบแขกที่ไม่มีต้องถูกปฏิเสธ	ผ่าน
11	test_empty_and_sorted_listing	กรณีไม่มีแขกเลย (K=0): list_rooms ต้องได้ลิสต์ว่าง และผลรวมโหลดต้องเป็น 0 (ไม่ error)	ผ่าน
12	test_group_all_or_nothing	กฎเพิ่มแขกเป็นกลุ่ม: ถ้ามีรหัสซ้ำแม้ 1 รายการ ต้องปฏิเสธทั้งกลุ่ม	ผ่าน
13	test_room_unchanged_after_ring_change	เทสหลักที่เชื่อมกับวงแหวน: หลังเพิ่มอาคาร D	ผ่าน
14	test_two_way_lookup	ดัชนีค้นหา 2 ทางต้องสอดคล้องกัน: รหัสแขก -> (อาคาร, ห้อง) แล้วเอาผลนั้นไปค้นกลับ	ผ่าน
คลาส TestPart3_BuildingManager			
15	test_add_building_moves_only_to_new_building	เพิ่มอาคาร N5: ผู้ย้ายต้องเข้า N5 เท่านั้น (4.4 ข้อ 2) จำนวนย้ายนับเป็นรหัสแขก	ผ่าน
16	test_add_then_remove_restores_original	เพิ่ม N5 แล้วลบ N5 วงแหวนกลับเหมือนเดิม แขกทุกคนต้องกลับไปอาคารเดิม	ผ่าน

#	เทส	สิ่งที่ตรวจ (บรรทัดแรกของ docstring)	ผล
17	test_edge_cases_leave_everything_unchanged	กรณีขอบ 4.5: เพิ่มซ้ำ, ลบที่ไม่มี, รหัสว่าง ต้องถูกปฏิเสธ (ok=False) โดยวงแหวนและแขกไม่เปลี่ยน	ผ่าน
18	test_expected_value_and_hash_ring_beats_modulo	ใช้ชุดแขกเดียวกัน เพิ่มอาคารเหมือนกัน: Consistent Hashing ย้ายน้อยกว่า hash mod N มาก	ผ่าน
19	test_move_records_and_room_lookup	รายการย้ายต้องมี รหัสแขก อาคารเดิม อาคารใหม่ เลขห้อง ที่ถูกต้อง	ผ่าน
20	test_no_guests_rate_not_computed	K=0 (4.4 ข้อ 4): จำนวนย้ายเป็น 0, ไม่คำนวณอัตราย้าย (None) และมีข้อความอธิบาย ไม่ error	ผ่าน
21	test_remove_building_moves_only_its_guests	ลบอาคาร N2: ย้ายเฉพาะแขกของ N2 (เท่ากับจำนวนแขกใน N2 ก่อนลบ) และไม่มีใครค้างที่ N2	ผ่าน
22	test_timing_fields_present	เก็บเวลาแยกตามขั้นตอน (ปรับวงแหวน / หาแขกที่ย้าย / ย้าย และปรับดัชนี) เป็นค่าไม่ติดลบ	ผ่าน
คลาส TestPart5_AdditionalRequirements			
23	test_csv_export_format	3. การ Export CSV (จำนวนแถวเท่ากับ K, Header ถูกต้อง)	ผ่าน
24	test_load_statistics	1. load_statistics: K=0 ได้ cv=None, กรณีปกติได้สถิติตามจริง , ลิสต์ว่างต้อง raise ValueError	ผ่าน
25	test_menu_compare_and_performance	6. ทดสอบการเปรียบเทียบ Consistent vs Modulo ในเมนู และการวัดประสิทธิภาพ	ผ่าน
26	test_menu_export_csv_action	5. ทดสอบว่า do_add_building บันทึก last_change_result และ do_export_csv เขียนไฟล์ทั้งสองได้จริง	ผ่าน
27	test_menu_input_handling_and_mock	4. เมนูจัดการอินพุตและดักจับข้อผิดพลาดผ่าน unittest.mock.patch	ผ่าน
28	test_sha256_fixed_values	2. ค่า SHA-256 64-bit ที่ fix ไว้เพื่อป้องกันพฤติกรรมแฮชเปลี่ยนข้ามเวอร์ชัน/การรัน	ผ่าน
คลาส TestSection9_2_MinimumChecklist			
29	test_9_2_1_new_guests_never_move_existing_guests	9.2 ข้อ 1: เพิ่มแขกใหม่ทั้งช่องทางเดิมและช่องทางใหม่ (รายคนและเป็นกลุ่ม)	ผ่าน
30	test_9_2_3_rejected_batch_leaves_data_untouched	9.2 ข้อ 3: กลุ่มที่ซ้ำเพียงบางรายการต้องถูกปฏิเสธ และข้อมูลเดิมต้องเหมือนเดิมทุกค่า	ผ่าน
31	test_9_2_6_csv_rows_match_current_guest_count	9.2 ข้อ 6: จำนวนแถวใน CSV ที่เมนูส่งออก = จำนวนแขกปัจจุบัน (หลังเพิ่ม/ลบแขกและเพิ่มอาคาร)	ผ่าน
32	test_9_2_6_remove_last_building_without_guests	4.5: ลบอาคารสุดท้ายต้องถูกปฏิเสธแม้ไม่มีแขก และวงแหวนยังค้นหาได้	ผ่าน
33	test_init_with_different_guest_count_per_channel	หัวข้อ 5 ข้อ 1: กำหนดจำนวนแขกของแต่ละช่องทางแยกกันได้ ลำดับ s เริ่มที่ 1 ทุกช่องทาง	ผ่าน
34	test_load_report_with_no_guests	หัวข้อ 5 ข้อ 7 / 7.2: K=0 รายงานทุกอาคารเป็น 0 สถิติพื้นฐานเป็น 0 และไม่คำนวณ CV	ผ่าน

กรณีที่ยังไม่ผ่าน: ไม่มี (0 ข้อ) ข้อจำกัดของการทดสอบ: `test_unique` ตรวจ Cantor เฉพาะ c, s ∈ [1, 59]; การพิสูจน์ทั่วไปอยู่ในหัวข้อ 3.2 และการทดสอบ CSV ในหัวข้อ 9.2 ข้อ 6 ตรวจไฟล์ที่เมนูส่งออก ส่วนไฟล์ results/*.csv ของการทดลองตรวจด้วย assert ใน experiments.py ขณะรัน (เช่น จำนวนแถว sample_guests.csv = K) ภาพกราฟอย่างเดียวไม่ได้ใช้เป็นหลักฐานความถูกต้อง

10. ข้อจำกัด

- **หาแขกที่ต้องย้ายแบบ ก (วนทั้ง K คน):** ทุกการเพิ่ม/ลบอาคารมีต้นทุน O(K log M) แม้ผู้ย้ายเพียงไม่กี่คน (ในการทดลอง ราว 12–13 ms ที่ K = 10,000) แบบ ข ที่ดูเฉพาะช่วงของ virtual node ทำให้น้อยกว่ามาก แต่ไม่ได้ทำ (เป็นงานเพิ่มเติม ไม่บังคับ)
- **การแทรก virtual node ใช้ bisect.insort ลง list:** O(V·M) เพราะต้องเลื่อนสมาชิก; ไม่เป็นปัญหาที่ M หลักร้อย แต่จะโตเร็วถ้า N และ V ใหญ่มาก (เช่น M ระดับแสน) ควรเปลี่ยนเป็นโครงสร้างต้นไม้/skip list หรือรวมการแทรกเป็นการ merge ครั้งเดียว
- **ตรวจความสอดคล้อง (verify_consistency) ทุกครั้งที่เปลี่ยนอาคาร:** เพิ่มการวนแขกอีก K คน O(K log M) ทำให้ต้นทุนรวมของการเปลี่ยนอาคารสูงกว่าเวลาที่รายงานใน t_update_data (ซึ่งไม่รวมส่วนนี้)
- **หน่วยความจำโตตาม K:** ราว 400–440 ไบต์ต่อแขก ตามที่ tracemalloc ติดตามได้ (K = 100,000 ใช้ current 42.10 MiB, peak 57.44 MiB) และตัวเลขนี้เป็นหน่วยความจำที่ Python ติดตาม ไม่ใช่ RSS ของโปรเซส
- **การเพิ่มแขกกลุ่มใหญ่มาก:** เมธอด `add_guests` สร้างรายการรหัสทั้งหมดและ set ตรวจซ้ำในหน่วยความจำพร้อมกัน O(B) ก่อนเพิ่ม (เพื่อให้เป็น all-or-nothing)
- **ไม่มีการเก็บถาวรหรืออ่านข้อมูลกลับจากไฟล์:** ข้อมูลอยู่ในหน่วยความจำ ปิดโปรแกรมแล้วหาย; CSV เป็นเพียงไฟล์ส่งออก (ไม่มีฟังก์ชันนำเข้า)
- **ส่วนติดต่อเป็นเมนูข้อความเท่านั้น** ไม่มี GUI ไม่มีภาพเคลื่อนไหวหรือภาพวงแหวน
- **การทดลอง:** ใช้เพียง 3 รอบ (salt 0, 1, 2) ต่อค่าพารามิเตอร์ จึงบอกความแปรปรวนได้เพียงคร่าวๆ ชุดแขกเป็นแบบสม่ำเสมอ (10 ช่องทาง จำนวนเท่ากัน) ไม่ใช่การกระจายจริง; salt เปลี่ยนเฉพาะตำแหน่งแฮช ชุดแขกและอาคารเหมือนเดิม; เวลาเป็นผลบนเครื่องเดียว; ในการทดลอง B ลบ NODE01 เสมอ (ซึ่งเป็นกรณีที่ไม่ดีสำหรับ hash mod N เพราะดัชนีเลื่อนทั้งหมด)
- Cantor pairing ให้เลขห้องโตแบบกำลังสองของ (c + s) เลขห้องจึงใหญ่ได้เมื่อ c หรือ s มาก (Python รองรับได้ แต่ต้นทุนการคำนวณโตตามความยาวของจำนวนเต็ม)

11. ตารางหน้าที่สมาชิก

สัดส่วนงานและส่วนที่พัฒนา/ทดสอบอ้างอิงตามตารางใน README.md ของโปรเจกต์ (สัดส่วนเท่ากันคนละ 20%)

ชื่อ-นามสกุล	รหัสนักศึกษา	สัดส่วน	ส่วนที่พัฒนาและทดสอบ	ลายมือชื่อ
ณัฐภูมิ บาดขุนทด	68010366	20%	Part 1: hash_ring.py (HashRing, compute_sha256_64bit, ModuloNodeSelector); ทดสอบ TestPart1_HashRingCore (รวมตัวอย่างวงแหวนเล็ก 9.1); จัดทำรายงาน	
ฑริตา เปรมชุติวัต	68010283	20%	Part 2: guest_room.py (Guest, cantor_room_no, GuestManager, ดัชนี 2 ทาง); ทดสอบ TestPart2_CantorRoomNo, TestPart2_GuestManager	
ปลิตา ชูรัตน์	68010669	20%	Part 3: building_manager.py (BuildingManager, MoveRecord, BuildingChangeResult, ตรวจกติกาการย้ายและกรณีขอบ); ทดสอบ TestPart3_BuildingManager	
พงศธร ฟักผล	68010726	20%	Part 4: common.py, experiments.py, plots.py (ออกแบบการทดลอง A, B, C วัดเวลา/หน่วยความจำ ส่งออก CSV วาดกราฟ 3 ภาพ); จัดทำรายงาน	
ปริญา บุญสิงมา	68010662	20%	Part 5: menu.py (เมนูโต้ตอบ ส่งออก CSV เปรียบเทียบและวัดประสิทธิภาพ); ทดสอบ TestPart5_AdditionalRequirements; จัดทำ README.md	
รวม		100%		

12. แหล่งอ้างอิง

แหล่ง	นำไปใช้อย่างไร
โจทย์โครงงาน OODS 2569 เรื่อง Distributed Hilbert's Hotel (เอกสารรายวิชา)	กติกาการคำนวณ (หัวข้อ 4), ตัวอย่างตรวจสอบ 4.2 และ 9.1, รูปแบบการทดลอง 7, แนวการวิเคราะห์ Big-O 8, เกณฑ์ 11
ทบทวนปัญหา Hilbert's Hotel: https://youtu.be/HLTjDXT9SqQ	ทำความเข้าใจที่มาของปัญหา “แขกทุกคนต้องย้าย” และแรงจูงใจของการคำนวณห้องจากรหัสแขก
Consistent Hashing (ByteByteGo): https://www.youtube.com/watch?v=UF9Iqmg94tk	แนวคิดวงแหวนแฮช การเลือกอาคารเจ้าของจุดแรกตามเข็มนาฬิกา และ virtual node เพื่อสมดุลโหลด
Python bisect: https://docs.python.org/3/library/bisect.html	`bisect_left` ค้นอาคารบนวงแหวน และ `insort` แทรกจุด (วิเคราะห์ต้นทุนการแทรก O(M) ต่อจุด)
Python tracemalloc: https://docs.python.org/3/library/tracemalloc.html	วัดหน่วยความจำ current/peak (ส่วนที่เพิ่มจากจุดเริ่มต้น)
Python time.perf_counter: https://docs.python.org/3/library/time.html#time.perf_counter	จับเวลาแต่ละขั้นตอนและการค้นหาเป็นชุด 1,000 ครั้ง